



SISTEMAS OPERACIONAIS

DIRIGENTES

PRESIDÊNCIA

Prof. Dr. Clèmerson Merlin Clève

REITORIA

Prof. Me. Alessandro Kinal

DIRETORIA ACADÊMICA EAD

Profa. Me. Daniela Ferreira Correa

DIRETORIA ACADÊMICA PRESENCIAL

Profa. Me. Márcia Maria Coelho

DIRETORIA EXECUTIVA

Profa. Esp. Silmara Marchioretto

COORDENAÇÃO PEDAGÓGICA DE GRADUAÇÃO EAD

Prof. Me. João Marcos Roncari Mari

COORDENAÇÃO PEDAGÓGICA DE PÓS-GRADUAÇÃO EAD

Prof. Me. Marcus Vinícius Roncari Mari

FICHA TÉCNICA

AUTOR

Prof.a Ma. Janine Donato Spinardi

COORDENAÇÃO DA PRODUÇÃO DE MATERIAIS EAD

Esp. Janaína de Sá Lorusso

PROJETO GRÁFICO

Esp. Janaína de Sá Lorusso

Esp. Cinthia Durigan

DIAGRAMAÇÃO

Esp. Janaína de Sá Lorusso

REVISÃO

Esp. Ísis C. D'Angelis

Esp. Idamara Lobo Dias

PRODUÇÃO AUDIO VISUAL

Esp. Rafael de Farias Forte Canonico

Estúdio NEAD (Núcleo de Educação a Distância) - UniBrasil

ORGANIZAÇÃO

NEAD (Núcleo de Educação a Distância) - UniBrasil

IMAGENS

Shutterstock



EDIÇÃO: AGO/2021

SUMÁRIO

UNIDADE 01 - CONCEITUANDO SISTEMAS OPERACIONAIS

OBJETIVOS DE APRENDIZAGEM	6
INTRODUÇÃO.....	7
1. OPERACIONAL	7
1.1 Breve Histórico dos Computadores	7
1.2 Conhecendo os principais conceitos.....	9
1.3 Tipos de Sistemas Operacionais	11
1.4 Interrupções e Exceções.....	12
2. FUNÇÕES E ESTRUTURA DE UM SISTEMA OPERACIONAL	13
2.1 Funções de um Sistema Operacional.....	13
2.2 Estrutura de um Sistema Operacional	14
3. PROCESSOS	15
3.1 Conceitos básicos	15
3.2 Comunicação e Sincronização	17
3.3 Escalonamento	18
CONSIDERAÇÕES FINAIS	19

UNIDADE 02 - GERENCIANDO SISTEMAS OPERACIONAIS

OBJETIVOS DE APRENDIZAGEM	20
INTRODUÇÃO.....	21
1. GERENCIAMENTO DE MEMÓRIA.....	21
1.1 Tipos de Memória.....	22
1.2 Memória Lógica e Física	23
1.3 Partições Fixas e Variáveis	23
1.4 Realocação	25

SUMÁRIO

1.5	Memória virtual	25
1.6	Swapping.....	28
2.	GERENCIAMENTO DE SISTEMAS DE ARQUIVOS.....	28
2.1	Arquivos	28
2.2	Diretórios	31
2.3	Cache de Sistemas de Arquivos.....	32
2.4	Gerência do Espaço Livre	33
3.	GERENCIAMENTO DE DISPOSITIVOS DE ENTRADA/SAÍDA	33
3.1	Interrupções	35
3.2	Software de Entrada e Saída	36
	CONSIDERAÇÕES FINAIS	36
UNIDADE 03 - SISTEMAS OPERACIONAIS DE MERCADO E REDUZIDOS		
	OBJETIVOS DE APRENDIZAGEM.....	38
	INTRODUÇÃO.....	39
1.	SISTEMAS OPERACIONAIS DE MERCADO	39
1.1	Windows	40
1.2	Linux	44
1.3	Diferenças entre Windows e Linux.....	46
1.4	MAC OSX	46
1.5	z/OS	47
2.	SISTEMAS OPERACIONAIS REDUZIDOS.....	48
2.1	iOS.....	48
2.2	Android	49
	CONSIDERAÇÕES FINAIS	51

SUMÁRIO

UNIDADE 04 - MÁQUINAS VIRTUAIS, COMANDOS SHELL E IMPLEMENTAÇÃO DE SCRIPTS

OBJETIVOS DE APRENDIZAGEM.....	52
INTRODUÇÃO.....	53
1. MÁQUINAS VIRTUAIS	53
1.1 Breve histórico.....	54
1.2 Aplicações para Máquinas Virtuais	54
1.3 Vantagens e Desvantagens de uma VM.....	56
1.4 Virtualização.....	57
1.5 VMware Workstation.....	59
2. LINGUAGEM DE COMANDOS E SCRIPTS	60
2.1 Scripts	61
3. ESTRUTURA DOS SISTEMAS OPERACIONAIS.....	63
CONSIDERAÇÕES FINAIS	65
REFERÊNCIAS	66

UNIDADE

01

CONCEITUANDO SISTEMAS OPERACIONAIS



OBJETIVOS DE APRENDIZAGEM

- » Compreender os conceitos relacionados a um sistema operacional;
- » Estudar a respeito das funções e da estrutura de um sistema operacional;
- » Entender como funcionam os processos nos sistemas operacionais.



VÍDEOS DA UNIDADE



<https://bit.ly/3wYXP3d>



<https://bit.ly/3ydGhBG>



<https://bit.ly/3hZEXg6>

INTRODUÇÃO

Nesta Unidade vamos estudar sobre os principais conceitos relacionados a sistemas operacionais, onde iremos compreender o que é um sistema computacional, quais seus componentes e onde se encaixam os sistemas operacionais, além de conhecermos um pouco do histórico dos computadores.

O sistema operacional nada mais é que uma camada de software (parte lógica) que opera entre o hardware (parte física) e os softwares aplicativos e também proporciona uma melhor interface com o usuário do sistema.

Também iremos estudar sobre os tipos de sistemas operacionais existentes no mercado, como os sistemas operacionais monoprogramáveis e multiprogramáveis, além dos sistemas que trabalham com mais de um processador.

Será possível estudarmos quais são as funções de um sistema operacional e como se compõem sua estrutura, além de entender como funcionam os processos.

Para que um computador possa ser utilizado é necessário termos um sistema operacional para que suas funções sejam iniciadas, ele é responsável pelo gerenciamento de dispositivos de entrada e saída, pelo gerenciamento de memória, além de ser necessário para que os softwares aplicativos possam ser utilizados.

Temos sistemas operacionais que podem ser utilizados em computadores pessoais, computadores nas organizações, dispositivos móveis, entre outros.

A partir desta Unidade será possível compreender os conceitos relacionados a sistemas operacionais e compreender seu funcionamento.

Os objetivos desta unidade são compreender os conceitos relacionados a um sistema operacional; estudar a respeito das funções e da estrutura de um sistema operacional e entender como funcionam os processos nos sistemas operacionais.

1. SISTEMA OPERACIONAL

Neste tópico vamos iniciar nossos estudos sobre sistemas operacionais, começando por um breve histórico dos computadores e depois compreendendo os principais conceitos relacionados aos sistemas operacionais.

1.1 BREVE HISTÓRICO DOS COMPUTADORES

Para compreendermos como os computadores surgiram e evoluíram, vamos retomar de forma breve o seu histórico, baseados nos estudos de Alves (2014).

No período da Segunda Guerra Mundial, surgiram os primeiros computadores desenvolvidos

nos EUA e na Inglaterra com objetivo militar, para realizar cálculos e decifrar códigos. Esses computadores eram baseados em relés eletromecânicos e chaves interruptoras.

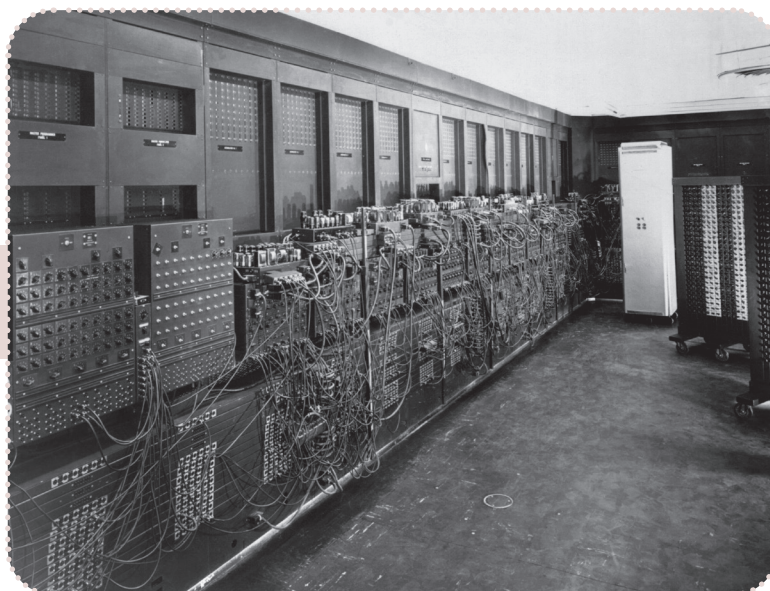
Na sequência um dispositivo com funcionamento eletrônico foi desenvolvido. O primeiro computador eletrônico que funcionava com válvulas, o ENIAC (*Electrical Numerical Integrator and Calculator*), era gigantesco, pois ocupava uma sala de 9 m X 30 m e ainda gerava um calor equivalente a 200 kW. Sua construção começou em 1943, mas somente em 1946 ele foi oficialmente apresentado, depois de terminada a Segunda Guerra Mundial.

Para programar tal computador eram utilizadas chaves (interruptores) manuais e a entrada dos dados era feita por meio de cartões perfurados. Esse foi o mais famoso computador da época, porém existiram outros antes dele.

A partir de 1947 com o surgimento do transistor, foi possível criar computadores mais baratos. Este foi a base para a evolução da microeletrônica e logo da computação moderna. Assim foram criados computadores menores e mais confiáveis, surgindo a segunda geração de computadores.

Mais adiante, a terceira geração de computadores foi marcada devido ao surgimento do circuito integrado, desenvolvido em 1958. E na sequência com a revolução da microeletrônica foi possível a miniaturização dos circuitos integrados, surgindo o microprocessador e com ele a quarta geração de computadores.

FIGURA 1 - ENIAC



Alves (2014), divide os avanços mais importantes da computação em 5 gerações, conforme apresentado a seguir:

- » 1ª Geração (entre 1940 e 1955): invenção da válvula e desenvolvimento de computadores eletromecânicos com fins militares;

- » 2ª Geração (entre 1955 e 1965): utilização de transistores em substituição às válvulas termiônicas, processamento em lote (batch) e surgimento de linguagens de programação de alto nível, como COBOL, FORTRAN e ALGOL. Esses computadores eram chamados de mainframes e os programas eram transferidos por meio de cartões perfurados;
- » 3ª Geração (entre 1965 e 1980): utilização de circuitos integrados, neste período surgiu o computador IBM System/360 e o sistema operacional OS/360, além do emprego de técnicas de multiprogramação e *time sharing*;
- » 4ª Geração (entre 1980 e 1990): surgimento dos computadores pessoais, a partir da evolução do microprocessador, também surgiram os primeiros sistemas operacionais de disco, como por exemplo: Apple DOS, TRS-DOS, MS-DOS, PC-DOS, entre outros e os sistemas operacionais de rede, como o Novell Netware;
- » 5ª Geração (a partir da década de 1990): surgiram os primeiros sistemas operacionais com interface gráfica e suporte a multitarefa, houve a popularização do modelo cliente/servidor e posteriormente surgiram sistemas operacionais para dispositivos móveis.

A partir deste breve histórico poderemos conhecer os principais conceitos relacionados à computação e a sistemas operacionais.



VÍDEO

Para conhecer mais sobre a história dos sistemas operacionais, assista ao vídeo no link a seguir que traz um breve resumo do histórico: <https://bit.ly/3xFEbui>. Acesso em 15 jun. 2021.

1.2 CONHECENDO OS PRINCIPAIS CONCEITOS

Para iniciarmos nosso estudo a respeito de Sistemas Operacionais, primeiramente vamos compreender como funciona um sistema computacional. Podemos dizer que um sistema computacional é composto por um hardware que processa informações de acordo com um software.

O hardware é composto pelas partes físicas de um sistema computacional, sendo elas: a Unidade Central de Processamento - (UCP/CPU – *Central Processing Unit*), a memória e os dispositivos de entrada e saída.

A CPU é responsável por processar os dados, executar operações e cálculos, possibilita o processamento de dados em alta velocidade. A memória se divide em memória primária ou principal e secundária. A memória primária (volátil) contém as informações necessárias para o processador em um dado momento, ou seja, durante a execução de um programa, por exemplo. A memória secundária (não volátil) guarda os dados de forma permanente, como por exemplo, nos HDs (disco rígido), SSDs, pendrives, DVDs entre outros.

Em relação aos dispositivos de entrada e saída, temos diversos, como exemplos de dispositivos de entrada podemos citar: mouse, teclado, microfone, leitor de código de barras, entre outros. E como dispositivos de saída podemos citar: impressora, monitor, caixas de som, entre outros. Ainda temos dispositivos que fazem tanto a entrada quanto a saída de dados, como por exemplo, monitor *touch screen*.

FIGURA 2 - DISPOSITIVOS DE ENTRADA E SAÍDA



Em relação ao software, podemos dizer que ele é a parte lógica do sistema de informação, ou seja, são os programas. Podemos aqui citar os sistemas operacionais e softwares aplicativos.

Segundo Capron (2004, p. 34), software é um “conjunto de instruções planejadas, passo a passo, necessárias para transformar dados em informação – que torna um computador útil.” Como dados podemos considerar os dados brutos e a informação seriam os dados com significado, já processados.

Podemos considerar que o sistema operacional tem como objetivo controlar o hardware e coordenar seu uso pelos diversos softwares aplicativos de vários usuários (SILBERSCHATZ; GALVIN; GAGNE, 2015).

Para Machado e Maia (2017), o sistema operacional é um conjunto de rotinas que é executado pelo processador, tendo como principal função controlar o funcionamento de um computador e também, permitindo gerenciar e compartilhar seus recursos, como por exemplo, processadores, memórias e dispositivos tanto de entrada quanto de saída. Ele pode ser considerado como uma camada de software que fica entre o hardware os outros programas, como os aplicativos.

Conforme cita Alves (2014, p. 25),

ANTES DO SURGIMENTO DOS SISTEMAS OPERACIONAIS, QUANDO A ERA DA COMPUTAÇÃO AINDA ENGATINHAVA COM OS PRIMEIROS COMPUTADORES, OS PROGRAMADORES PRECISAVAM CRIAR SEUS PRÓPRIOS CÓDIGOS PARA A EXECUÇÃO DE TAREFAS ELEMENTARES, COMO LEITURA OU ESCRITA DE DADOS.

Ou seja, sem o sistema operacional seria uma tarefa muito difícil para que um computador funcionasse e fizesse operações, pois o usuário precisaria ter um conhecimento muito profundo a respeito da parte de hardware. O sistema operacional permite que a utilização de um computador seja feita de forma fácil e bastante intuitiva, principalmente nos sistemas operacionais que utilizam recursos gráficos, como o Windows. O sistema operacional permite gerenciar diversas atividades, conforme vamos compreender em nossos estudos.

Já os softwares aplicativos possuem uma função específica, como por exemplo, um editor de textos. Os aplicativos precisam de um sistema operacional para que eles funcionem, seja em um computador, ou um dispositivo móvel, por exemplo.

Outro termo que podemos utilizar é o firmware, que é uma combinação entre hardware e software. Eles estão presentes nos computadores na forma de BIOS, leitores ou gravadores de CDs, por exemplo.

1.3 TIPOS DE SISTEMAS OPERACIONAIS

De acordo com as características e a evolução, podemos classificar os sistemas operacionais em monoprogramáveis ou monotarefas, multiprogramáveis ou multitarefas e com múltiplos processadores.

1.3.1 SISTEMAS OPERACIONAIS MONOPROGRAMÁVEIS

Os sistemas operacionais monoprogramáveis também são conhecidos por monotarefas. Quando os sistemas operacionais surgiram eles possuíam a característica de executar apenas um programa ou tarefa de cada vez, sendo que para executar um novo programa era necessário que o primeiro fosse finalizado ou interrompido. Desta forma, tanto o processador, quanto a memória e os dispositivos periféricos ficavam dedicados à execução de uma única tarefa.

Esse tipo de sistema operacional está relacionado aos computadores que surgiram na década de 1960 e permaneceu sendo utilizado durante um bom tempo.

1.3.2 SISTEMAS OPERACIONAIS MULTIPROGRAMÁVEIS

Os sistemas operacionais multiprogramáveis também são conhecidos como multitarefas, pois neste tipo de sistema operacional os recursos são compartilhados. Esse compartilhamento pode ocorrer entre vários usuários ou entre várias aplicações.

Neste caso, podemos executar diversas tarefas ao mesmo tempo, por exemplo, enquanto efetuamos a impressão de um documento redigido em um editor de textos, podemos assistir ou editar um vídeo. Sendo assim, os recursos como memória e processador são compartilhados.

Já que os programas compartilham o uso da memória é necessário que cada usuário tenha uma área reservada, como mecanismo de proteção. Ou seja, se o programa tentar acessar uma posição fora de sua área, haverá um erro de violação de acesso. O sistema operacional, nestes casos, precisa possuir mecanismos para realizar a comunicação de forma sincronizada e controlada, para que não haja problemas de inconsistência. Um disco também pode armazenar arquivos de diferentes usuários, sendo assim, o sistema operacional precisa garantir a confidencialidade de dados (MACHADO; MAIA, 2017).

Esse tipo de sistema operacional reduz o tempo de resposta de aplicações e consequentemente de custos computacionais. Segundo Pereira, Vissotto e Franciscatto (2015), os sistemas operacionais multiprogramáveis podem ser divididos em: sistemas batch, de tempo compartilhado e tempo real.

- » Sistemas batch: esses sistemas foram implementados na década de 1960 e não exigiam a interação do usuário. Nesta época, os programas eram submetidos para execução por meio da utilização de cartões perfurados, armazenados em discos ou fitas, para serem executados posteriormente, de acordo com a disponibilidade de memória principal;
- » Sistemas de tempo compartilhado: esses sistemas eram conhecidos como *time-sharing*, permitem que diversos programas sejam executados a partir da divisão do tempo do processador em fatias de tempo (*time slice*). Se a fatia de tempo for pequena, ele irá aguardar uma nova fatia de tempo para entrar em execução novamente. Esse tipo de sistema permite que os usuários interajam com o sistema por meio dos dispositivos de entrada de dados e comandos especiais;
- » Sistemas de tempo real: são conhecidos como *real time* e têm características semelhantes aos de tempo compartilhado. Diferente dos sistemas de tempo compartilhado, neste caso, o processador permanece ocupado durante o tempo necessário para executar um programa e só cede lugar a outro que tenha maior prioridade.

1.3.3 SISTEMAS OPERACIONAIS COM MÚLTIPLOS PROCESSADORES

Temos também, os sistemas operacionais que trabalham com múltiplos processadores, ou seja, têm duas ou mais unidades centrais de processamento que trabalham de forma integrada. Neste sistema, é possível termos vários softwares trabalhando ao mesmo tempo, as tarefas são executadas de forma simultânea. Também é possível que um programa seja dividido em partes para serem processadas ao mesmo tempo em mais de um processador.

Atualmente é muito comum utilizarmos computadores que possuem mais de uma unidade central de processamento. O termo multiprocessamento está relacionado à capacidade do sistema suportar mais processadores e também de alocar as tarefas nestes vários processadores.

Este tipo de sistema compartilha o mesmo espaço de endereçamento de memória e é gerenciado pelo sistema operacional que possibilita termos filas de processos diversas, sendo uma para cada processador.

O que diferencia um sistema com múltiplos processadores de um multitarefa é que a multitarefa utiliza recursos como o compartilhamento de tempo para utilizar o processador.

1.4 INTERRUPÇÕES E EXCEÇÕES

No momento que um programa está sendo executado podem ocorrer eventos que forcem a parada da execução. Neste caso, entram as interrupções e as exceções, que podem acontecer em consequência da sinalização de algum dispositivo de hardware ou da execução de instruções de um programa.

As interrupções são causadas por um evento externo ao programa que está sendo executado. Estas são advindas de eventos assíncronos, não estão relacionadas com a instrução do programa que está executando neste momento.

As exceções são semelhantes às interrupções, porém ela pode ocorrer por meio de uma instrução do próprio software. Estas são geradas por um evento síncrono, direto da execução do programa corrente.



SAIBA MAIS

Sistemas Operacionais mais usados

No mundo do software, quatro sistemas operacionais importantes continuam a ser populares. São iOS, Android, Windows e MacOS. Então, qual é o sistema operacional mais usado? Qual versão de sistema operacional é mais usada?

Qual sistema operacional é o mais usado?

Android, iOS, Windows e MacOS, que são os sistemas operacionais mais usados no mundo do computador e móvel, também têm certas taxas de uso. Quando olhamos para o sistema operacional Android do Google, vemos que a versão mais usada é o Android Nougat. O sistema operacional Nougat com uma taxa de 28,5% é seguido pelo Android 6 Marshmallow com 28,1%. A versão Android Orheo tem apenas 1,1% de taxa de uso.

Para saber mais leia o texto na íntegra em: <https://bit.ly/3BbSXv9>. Acesso em 15 jun. 2021.

2. FUNÇÕES E ESTRUTURA DE UM SISTEMA OPERACIONAL

Neste tópico, vamos conhecer sobre as funções básicas de um sistema operacional e mais adiante iremos detalhar tais funções. Além disso, vamos também, estudar a respeito da estrutura de um sistema operacional.

2.1 FUNÇÕES DE UM SISTEMA OPERACIONAL

Uma das principais funções de um sistema operacional é realizar o controle do computador, além de gerenciar e compartilhar os seus recursos, sejam eles: processador, memória e periféricos (entrada e saída).

Um sistema operacional se encarrega de gerenciar os processos que estão em execução em dado momento, gerencia a memória e os recursos computacionais. Além disso, encarrega-se de controlar os dispositivos de entrada e saída de dados. Também é o sistema operacional que gerencia os arquivos em nosso computador, ou seja, o sistema operacional é essencial para a realização de todas as tarefas a serem desenvolvidas em um sistema computacional.

Por meio do sistema operacional que conseguimos executar outros softwares, como por exemplo, softwares aplicativos para edição de textos, planilhas, bancos de dados, entre outros. Ele também se encarrega da interface com o usuário, permitindo o acesso de forma rápida e fácil a todos os recursos computacionais.

2.2 ESTRUTURA DE UM SISTEMA OPERACIONAL

Um sistema operacional executa um conjunto de rotinas que é chamado de núcleo do sistema (kernel). Tais rotinas acontecem com base em eventos assíncronos que muitas vezes são geradas pelo hardware.

Os usuários podem se comunicar com o núcleo do sistema por meio de rotinas do sistema que são realizadas por aplicações ou ainda por meio de utilitários ou linguagem de comandos, que são específicos de cada sistema operacional. (MACHADO; MAIA, 2017).

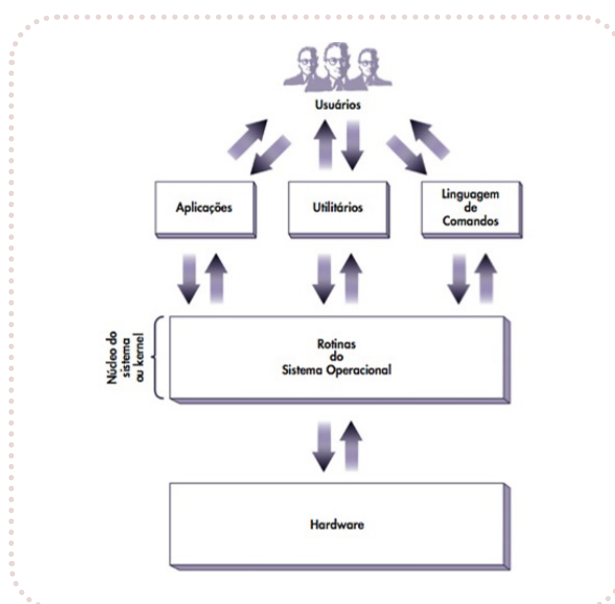
Machado e Maia (2011, p. 47), citam como algumas das principais funções do núcleo:

TRATAMENTO DE INTERRUPÇÕES E EXCEÇÕES; CRIAÇÃO E ELIMINAÇÃO DE PROCESSOS E THREADS; SINCRONIZAÇÃO E COMUNICAÇÃO ENTRE PROCESSOS E THREADS; ESCALONAMENTO E CONTROLE DOS PROCESSOS E THREADS; GERÊNCIA DE MEMÓRIA; GERÊNCIA DO SISTEMA DE ARQUIVOS; GERÊNCIA DE DISPOSITIVOS DE E/S; SUPORTE A REDES LOCAIS E DISTRIBUÍDAS; CONTABILIZAÇÃO DO USO DO SISTEMA; AUDITORIA E SEGURANÇA DO SISTEMA.

Algumas dessas funções serão tratadas mais adiante dentro desta disciplina.

Na figura a seguir, apresentamos um modelo de camadas detalhando a estrutura do sistema operacional e suas interfaces.

FIGURA 3 - ESTRUTURA DO SISTEMA OPERACIONAL



Fonte: Machado e Maia (2017).

Devido à complexidade da arquitetura de um sistema multiprogramável, podem surgir problemas de segurança no inter-relacionamento dos diversos subsistemas existentes. O sistema operacional controla os acessos e áreas reservadas a cada usuário. Também cuida para que um programa não acesse uma posição de memória fora de sua área, assim garante a integridade e confidencialidade de dados dos usuários. Ele deve controlar o acesso concorrente aos recursos do sistema.

Também temos sistemas operacionais simples que não têm uma estrutura bem definida, por exemplo, o MS-DOS. Neste sistema, as interfaces e níveis de funcionalidade não estão bem separados, assim fica vulnerável a programas errados ou maliciosos. Era limitado em sua época também devido ao hardware. Além do MS-DOS, o UNIX também foi limitado pelo hardware, inicialmente.

Em geral os sistemas operacionais combinam estruturas diferentes e resultam em sistemas híbridos, assim resolvem problemas de desempenho, segurança e usabilidade. Como exemplo de sistemas híbridos temos, o Mac OS X da Apple e os dois sistemas operacionais móveis iOS e Android (SILBERSCHATZ; GALVIN; GAGNE, 2015).



REFLITA

A partir da evolução dos computadores e sistemas operacionais, pesquise, analise e reflita sobre a importância da evolução tecnológica para os dias atuais.

3. PROCESSOS

Neste tópico vamos estudar sobre os processos, que é basicamente um programa que se encontra em execução, mas também, pode ser um conjunto de informações necessárias para a concorrência de programas em um sistema operacional.

3.1 CONCEITOS BÁSICOS

Uma das principais funções de um sistema operacional é o gerenciamento de processos o qual possibilita aos programas alocar recursos, compartilhar dados, trocar informações e sincronizar suas execuções, sendo que nos sistemas multiprogramáveis, estes processos são executados de forma concorrente, permitindo o compartilhamento de recursos de hardware. Um processo é formado pelas seguintes partes: contexto de hardware, contexto de software e espaço de endereçamento (MACHADO; MAIA, 2011).

O sistema operacional implementa a concorrência executando de forma alternada as instruções na unidade central de processamento. Quando um programa é interrompido as informações precisam ser guardadas para que ao retornar o seu processamento não falte nenhuma informação necessária à sua continuidade. Um processo ativo pode passar pelos estados de execução, pronto e espera.

Ele está no estado de execução quando está sendo processado pela unidade central de processamento. Se houver apenas um processador, somente um processo estará em execução em um determinado instante, mas se houver mais de um processador, poderemos ter vários processos sendo executados no mesmo momento. Se o processo estiver aguardando para ser executado, ele estará no estado de pronto e é o sistema operacional que irá gerenciar qual dos processos irá utilizar o processador naquele instante. Para realizar esta operação o processador utiliza o recurso de escalonamento. Já quando um processo aguarda que ocorra algum evento para dar continuidade em seu processamento, ele estará em estado de espera.

Em sistemas multiprogramáveis, ocorre o mecanismo chamado de concorrência, ou seja, o processador pode executar diversos programas ao mesmo tempo e estes são executados de forma concorrente pelo sistema operacional.

Machado e Maia (2011) afirmam que é possível implementar a concorrência de três formas diferentes, sendo elas: processos independentes, subprocessos e threads. Para os autores, o uso de processos independentes é a forma mais simplificada de implementar a concorrência em sistemas multiprogramáveis. Criar um processo independente exige a alocação de um bloco de controle de processo que possui contextos de hardware e software e espaços de endereçamento próprios.

Já no caso de subprocessos, os mesmos autores afirmam que estes são processos criados dentro de uma estrutura hierárquica, ou seja, um processo dentro de outro processo. Neste caso, existirá uma dependência entre o processo criador e o subprocesso, sendo assim, se o processo-pai deixar de existir, todos os subprocessos subordinados a ele também serão excluídos. Os subprocessos também possuem seu próprio bloco de controle de processo.

Os threads procuram reduzir o tempo utilizado na criação, eliminação e troca de contexto de processos nas aplicações concorrentes e assim, economizam recursos do sistema como um todo. Elas compartilham o processador assim como um processo e possuem seu próprio contexto de hardware, mas compartilham o mesmo contexto de software e espaço de endereçamento com os outros threads do processo. Compartilhando o espaço de endereçamento, a comunicação de threads dentro de um mesmo processo será realizada de forma simples e rápida.

Córdova Junior, Ledur e Moraes (2018), apresentam as seguintes diferenças entre os threads e os processos de um sistema operacional multitarefa tradicional:

- » Os processos são independentes e os threads existem como subconjuntos de um processo;
- » Os processos possuem mais informações de estado do que os threads. Diversos threads dentro de um processo compartilham o estado do processo, a memória e outros recursos;
- » Os processos têm espaços de endereço separados e os segmentos compartilham o espaço de endereço;
- » Os processos interagem por meio de mecanismos de comunicação entre processos fornecidos pelo sistema;
- » A alternância de contexto entre threads no mesmo processo é tipicamente mais rápida que a alternância de contexto entre processos.

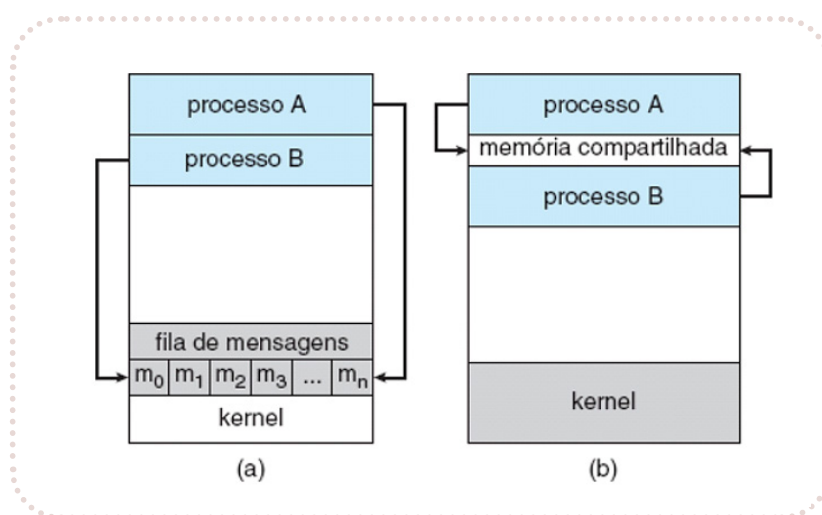
3.2 COMUNICAÇÃO E SINCRONIZAÇÃO

Os processos em um sistema operacional podem ser independentes ou cooperativos. É chamado independente quando não afeta outros processos em execução e nem é afetado por eles. Ele não compartilha dados com outros processos. Já o cooperativo pode afetar outros processos ou ainda ser afetado por eles.

Segundo Silberschatz, Galvin e Gagne (2015), os processos cooperativos precisam de um mecanismo de comunicação entre os processos (IPC), que permite trocar dados e informações. Temos dois modelos básicos de comunicação em processos: memória compartilhada e transmissão de mensagens. Veja a comparação desses modelos na figura a seguir.

- » Memória compartilhada: é estabelecida uma região da memória que é compartilhada por processos cooperativos, os quais podem trocar informações lendo e gravando nesta área;
- » Transmissão de mensagens: ocorre por meio de mensagens que são trocadas entre os sistemas cooperativos.

FIGURA 4 - MODELOS DE COMUNICAÇÃO



(A) TRANSMISSÃO DE MENSAGENS; (B) MEMÓRIA COMPARTILHADA.

Fonte: Silberschatz, Galvin e Gagne (2015).

A memória compartilhada pode ser mais rápida que a transmissão de mensagens, porém a transmissão de mensagens pode fornecer melhor desempenho. A comunicação entre processos que usam memória compartilhada requer que os processos em comunicação estabeleçam uma região de memória compartilhada (SILBERSCHATZ; GALVIN; GAGNE, 2015).

A troca de mensagens é um mecanismo de comunicação e sincronização entre processos. A sincronização garante a comunicação entre processos concorrentes, sendo que estes mecanismos são de extrema importância no projeto de sistemas operacionais multiprogramáveis (MACHADO; MAIA, 2017).

Na sincronização condicional o acesso ao recurso compartilhado exige que a sincronização dos processos esteja vinculada a uma condição de acesso, sendo que o processo fica bloqueado até que o recurso esteja disponível. O conceito de semáforos é um mecanismo de sincronização que permite implementar de forma simples a exclusão mútua e a sincronização condicional entre processos.

3.3 ESCALONAMENTO

Por meio do escalonamento de processos o sistema operacional consegue manter o processador ocupado, equilibrando o uso da CPU entre os diversos processos. Desta forma, é possível priorizar aplicações críticas e maximizar o uso do processador. Os processos competem pela utilização da CPU, por isso o sistema operacional utiliza o recurso de escalonamento.

O escalonamento de processos funciona de acordo com cada sistema operacional e temos sistemas de tempo compartilhado e sistemas de tempo real. Podemos ter problemas de escalonamento tanto no caso de processos quanto de threads.

Machado e Maia (2011) apresentam alguns critérios que devem ser considerados na política escalonamento:

- » Utilização do processador: em geral, o ideal é que o processador fique a maior parte do tempo ocupado;
- » Throughput: representa a quantidade de processos executados em um determinado intervalo de tempo e quanto maior o valor, maior o número de tarefas executadas em relação ao tempo;
- » Tempo de Processador ou Tempo de CPU: é o tempo que um determinado processo leva no estado de execução durante seu processamento;
- » Tempo de Espera: tempo total que o processo permanece na fila de pronto, esperando para ser executado;
- » Tempo de Turnaround: é o tempo que um processo leva desde sua criação até seu término. Neste caso, é levado em consideração todo o tempo gasto na espera para alocação de memória, espera na fila de pronto, processamento na CPU e na fila de espera, como nas operações de entrada e saída;
- » Tempo de Resposta: tempo decorrido entre uma requisição ao sistema ou à aplicação e o momento em que a resposta é exibida. Geralmente, o tempo de resposta não é limitado pela capacidade de processamento do sistema computacional, mas pela velocidade dos dispositivos de entrada e saída.

A política de escalonamento pode ser classificada em escalonamentos preemptivos e não preemptivos. Quando o sistema operacional pode interromper um processo em execução e o substitui por outro, chamamos de escalonamento preemptivo e são mais complexos. Este tipo de escalonamento pode ser interrompido pelo sistema operacional para que outro processo seja alocado na CPU, assim existe a possibilidade de priorização de aplicações em tempo real, por exemplo. Já o escalonamento não preemptivo, quando um processo está em execução, nenhum evento pode

ocasionar a perda do uso do processador, pois este só sairá do estado de execução quando terminar seu processamento ou se houverem instruções no código que causem a mudança de estado.

Ainda temos outras opções de escalonamento, como o escalonamento FIFO, escalonamento circular e por prioridades. Vamos conhecer um pouco sobre cada um deles a seguir.

No escalonamento FIFO (*First In First Out* – primeiro que entra, primeiro que sai), o processo que chega primeiro ao estado de pronto será executado. Funciona como uma fila, na qual os processos que vão chegando como “pronto”, entram no final da fila e vão sendo escalonados quando chegam no início da fila.

O escalonamento circular ou *Round Robin Scheduling* é semelhante ao FIFO, mas assim que um processo passa para o estado de execução, ele terá um tempo limite para usar o processador de forma contínua, o que chamamos de fatia de tempo – *time slice* ou ainda de quantum. Cada vez que um determinado processo é escalonado para execução, ele terá uma nova fatia de tempo. Neste caso a interrupção ocorre por tempo, sendo um escalonamento do tipo preemptivo.

No escalonamento por prioridade, o processador executa o processo que possui maior prioridade no estado de pronto. Neste tipo de escalonamento a perda do uso do processador ocorre assim que um processo de maior prioridade passa para o estado de pronto ou se ocorrer algum tipo de mudança voluntária para o estado de espera.

É possível também, utilizar o escalonamento circular com prioridade. Neste caso, um processo fica em execução até que termine seu processamento, passe voluntariamente para o estado de espera ou ainda, tenha uma preempção por tempo ou prioridade.

CONSIDERAÇÕES FINAIS

Nesta Unidade foi possível conhecermos sobre os conceitos iniciais de sistemas operacionais, sua arquitetura e funcionamento. Assim conhecemos os diferentes tipos de sistemas, como os sistemas operacionais monoprogramáveis, multiprogramáveis e com múltiplos processadores.

Iniciamos a unidade conhecendo o histórico dos computadores e sistemas operacionais, para que fosse possível compreender sua evolução. Também, estudamos a respeito dos conceitos relacionados com o tema da Unidade.

Depois conhecemos as funções e a estrutura de um sistema operacional. Neste momento, ainda não focamos em um sistema operacional específico, mas sim pudemos compreender seu funcionamento.

Além desses conceitos, compreendemos como um programa é processado e de que forma ocorre o escalonamento dos processos.

Os sistemas operacionais não estão disponíveis atualmente somente em computadores, mas temos sistemas operacionais em diversos outros equipamentos, como por exemplo, podem ser utilizados em semáforos, no controle de um sistema de uma aeronave, entre diversas outras aplicações. Esses sistemas são chamados de embarcados e possuem uma tarefa específica, podendo ser utilizados em computação em tempo real. O uso de recursos de hardware para estes sistemas também podem ser reduzidos.

UNIDADE

02

GERENCIANDO SISTEMAS OPERACIONAIS



OBJETIVOS DE APRENDIZAGEM

- » Compreender o funcionamento do gerenciamento de memória;
- » Entender como funciona o sistema de arquivos de um sistema operacional;
- » Compreender sobre o gerenciamento dos dispositivos de entrada e saída.



VÍDEOS DA UNIDADE



<https://bit.ly/3wVZiqV>



<https://bit.ly/3hVUNIO>



<https://bit.ly/2UKK4rl>

INTRODUÇÃO

Nesta Unidade, vamos estudar sobre aspectos de grande importância nos sistemas operacionais. Apesar de cada sistema operacional ter suas características e especificidades, vamos compreender em linhas gerais como funcionam o gerenciamento de memória, o gerenciamento de sistemas de arquivos e o gerenciamento de dispositivos de entrada e saída.

Sempre que vamos utilizar um computador, independente do sistema operacional que está instalado. Estamos em contato direto com essas funções relacionadas a memórias, a arquivos e aos dispositivos.

O gerenciamento de memória auxilia no melhor desempenho dos processos/tarefas que são executadas pelo sistema operacional, pois ele permite que tenhamos um controle sobre os diversos processos em um sistema multiprogramável, onde é possível executar várias atividades ao mesmo tempo. Por exemplo, posso realizar a impressão de um documento enquanto estou realizando a edição de um vídeo em outro software, sendo que o sistema operacional irá controlar o gerenciamento de memória para as duas tarefas.

O gerenciamento de arquivos e diretórios permite uma adequada organização em nosso sistema, assim podemos ter arquivos e pastas separados para cada programa e também os arquivos e pastas que criamos para nosso trabalho ou uso pessoal, tais como vídeos, imagens, planilhas, textos, entre diversos outros formatos de arquivos.

Já o gerenciamento de dispositivos de entrada e saída também poderá afetar as outras tarefas do nosso sistema, pois é necessário um adequado controle de qualquer tipo de dispositivo que temos. Os dispositivos de entrada/saída também são conhecidos como periféricos de entrada/saída. Como exemplos de dispositivos de entrada podemos citar o mouse e o teclado, como dispositivos de saída podemos citar o monitor de vídeo e a impressora e ainda temos dispositivos que permitem a entrada e saída de dados, como um monitor *touch screen* ou um pendrive.

Esta Unidade apresenta como objetivos de aprendizagem: compreender o funcionamento do gerenciamento de memória, entender como funciona o sistema de arquivos de um sistema operacional e compreender sobre o gerenciamento dos dispositivos de entrada e saída.

1. GERENCIAMENTO DE MEMÓRIA

Nos sistemas monoprogramáveis o gerenciamento de memória ocorria de forma mais fácil, já nos sistemas multiprogramáveis, existe uma complexidade maior.

O gerenciamento de memória procura manter o maior número de processos residentes na memória principal e isto permite compartilhar o processador. Quando não existe espaço livre na memória principal, os processos são transferidos para a memória secundária para liberar o espaço para outros processos, essa transferência é chamada de *swapping*. Quando temos um programa

maior que a memória física disponível é utilizada a técnica de memória virtual ou de **overlay**, que veremos mais detalhes adiante.

Podemos compreender um processo como “um programa que se encontra em execução, ou seja, que está no momento ocupando o processador para efetuar alguma tarefa” (ALVES, 2014, p. 61).

Na imagem abaixo, podemos ver os estados que podem ser atribuídos a processo, como o estado de pronto, de execução ou bloqueado.

FIGURA 1 - ESTADOS QUE PODEM SER ATRIBUÍDOS A UM PROCESSO

PRONTO	Nesse estado, o processo está pronto para ser executado, mas isso ainda não ocorre porque não chegou sua vez, pois há outros processos na frente.
EXECUÇÃO	O processo está no momento ocupando a CPU na execução de uma operação.
BLOQUEADO	O processo que está em execução é bloqueado porque precisa de um recurso do sistema que não se encontra disponível.

Fonte: Alves (2014, p. 65).

1.1 TIPOS DE MEMÓRIA

Em um sistema computacional temos a memória primária (principal) e a memória secundária.

A memória primária contém as informações em um determinado momento. Como memória primária temos a memória RAM, a memória ROM, registradores e cache.

A memória RAM (**Random Access Memory** - memória de acesso aleatório) é considerada uma memória volátil, ou seja, se desligarmos o computador ou houver uma queda de energia perdemos o que está na memória principal. Ela mantém as informações armazenadas temporariamente, ou seja, enquanto estamos trabalhando em um determinado arquivo ou programa, por exemplo. Aqui encontramos a memória estática (SRAM) e a dinâmica (DRAM).

A memória ROM (**Read-Only Memory** – memória somente de leitura) é uma memória não volátil e permite somente a leitura das informações previamente gravadas pelo fabricante. Os principais tipos de memória ROM que podemos citar são: PROM, EPROM, EEPROM e EAROM.

Os registradores são um tipo de memória temporária utilizada pelo processador durante o processamento de determinadas instruções.

A memória cache armazena partes da memória principal, utilizadas com frequência pelo processador. Ela funciona como um buffer entre os registradores e a memória RAM. O hardware verifica se os dados necessários em um determinado momento estão no cache, pois assim, se estiverem são recuperados de forma mais rápida. Caso não estejam no cache, são acessados pela memória RAM.

Em relação à memória secundária, ela é considerada não volátil, ou seja, não perdemos a informação se o computador não estiver ligado na energia ou bateria. As informações são manti-

das de forma permanente. Para usar as informações que estão gravadas neste tipo de memória, é necessário que elas sejam carregadas na memória principal para que então o processador possa utilizá-las. As memórias secundárias utilizam alguns tipos de tecnologia, como: magnéticas, ópticas e de estado sólido.

As memórias magnéticas são um tipo de tecnologia que realiza o processo de gravação por meio de um cabeça magnética, utilizando o princípio de polarização para identificar dados em uma superfície magnetizável. Como exemplos temos as antigas fitas de áudio, os antigos disquetes e os HDs (*Hard Disk*) que são utilizados até hoje.

As memórias ópticas armazenam os dados em uma superfície reflexiva que usa um feixe de luz (laser) para realizar a leitura dos dados. As informações são gravadas em sulcos de tamanho microscópico. Como exemplo desse tipo de tecnologia, podemos citar os CDs, DVDs e Blu-rays.

As memórias de estado sólido ou os famosos SSDs (*Solid State Disk*), utilizam a tecnologia flash e são usadas em substituição aos HDs, porém são bem mais rápidas e resistentes, mas também custam mais caro. Esse tipo de memória é composta por circuitos.



SAIBA MAIS

Para aprender mais sobre os tipos de memória ROM, acesse: <https://bit.ly/3yYud7o> e faça a leitura do texto. Acesso em 18 jun. 2021.

1.2 MEMÓRIA LÓGICA E FÍSICA

Podemos considerar em um sistema computacional a memória lógica e a memória física, sendo que a memória lógica é a qual um processo pode endereçar e acessar a partir de suas instruções e esses endereços são endereços lógicos. Já a memória física é a parte eletrônica do computador composta pelos circuitos integrados de memória.

Outro componente que temos é a MMU, ou seja, a unidade de gerência de memória, a qual provê os mecanismos que serão utilizados pelo sistema operacional para o gerenciamento de memória, mapeando os endereços lógicos que são gerados pelos processos nos endereços físicos correspondentes.

1.3 PARTIÇÕES FIXAS E VARIÁVEIS

Consideramos como partições as divisões do espaço de um disco rígido e isso permite, por exemplo, termos diferentes tipos de sistema operacional, um em cada partição. As partições podem ser fixas ou variáveis.

As partições fixas são utilizadas no gerenciamento de memória da seguinte forma:

A MEMÓRIA É PRIMEIRAMENTE DIVIDIDA EM UMA PARTE PARA USO DO SISTEMA OPERACIONAL E UMA PARTE PARA USO DOS PROCESSOS DE USUÁRIOS. A SEGUIR, A PARTE DOS USUÁRIOS É DIVIDIDA EM VÁRIAS PARTIÇÕES DE TAMANHOS DIFERENTES, PORÉM FIXOS. QUANDO UM PROGRAMA DEVE SER CARREGADO, É ESCOLHIDA UMA PARTIÇÃO AINDA LIVRE. OBVIAMENTE, A PARTIÇÃO DEVE TER UM TAMANHO IGUAL OU MAIOR QUE O PROGRAMA. CASO NÃO EXISTA UMA PARTIÇÃO LIVRE QUE SEJA GRANDE O BASTANTE PARA CONTER O PROGRAMA, ELE NÃO PODERÁ SER EXECUTADO NO MOMENTO. DEVERÁ ESPERAR ATÉ QUE UM DOS PROCESSOS EM EXECUÇÃO TERMINE, LIBERANDO UMA PARTIÇÃO DE TAMANHO SUFICIENTE.

- (OLIVEIRA; CARISSIMI; TOSCANI 2010, p. 157)

Na imagem a seguir podemos ver a memória dividida em quatro partições, cada uma com um determinado tamanho e ainda uma reservada para o sistema operacional.

FIGURA 2 - MEMÓRIA FÍSICA DIVIDIDA EM PARTIÇÕES FIXAS

MEMÓRIA FÍSICA
SISTEMA OPERACIONAL - 225 KBYTES
PARTIÇÃO 1 - 200 KBYTES
PARTIÇÃO 2 - 100 KBYTES
PARTIÇÃO 3 - 50 KBYTES
PARTIÇÃO 4 - 25 KBYTES

Fonte: Oliveira, Carissimi e Toscani (2010, p. 157).

Um problema que pode ocorrer no caso da utilização de partições fixas é que, é muito difícil um programa que necessita ser carregado, tenha o tamanho exato de qualquer partição. Normalmente, ele será carregado em uma partição maior do que aquela necessária, ocorrendo a fragmentação.

A fragmentação interna acontece em espaços livres e contíguos da memória onde são pré-alocados por processos e não possibilitam o uso por outros processos, já a fragmentação externa ocorre em espaços livre e contínuos, porém podem ser muito pequenos e assim não possibilitam a alocação de programas por processos.

No caso das partições variáveis, o tamanho é ajustado de forma dinâmica e assim pode atender às necessidades dos processos. Ao criar um processo, o sistema operacional busca por espaços de tamanho igual ou maior que o processo a ser executado. Para determinar em qual espaço livre da memória um programa será carregado para execução, o sistema operacional pode utilizar algumas estratégias. Assim, é possível diminuir o problema da fragmentação externa. Essas estratégias são: *best-fit*, *worst-fit* e *first-fit*.

Best-fit: neste caso, o sistema operacional irá escolher a melhor partição, onde o programa deixará o menor espaço sem utilização. Neste algoritmo, ficam as menores áreas livres, após a alo-

cação do programa, assim a tendência é que cada vez mais a memória fique com pequenas áreas não contíguas aumentando o problema da fragmentação.

Worst-fit: neste caso, o sistema operacional irá escolher a pior partição, onde o programa deixa o maior espaço sem utilização. Apesar de deixar espaços maiores é possível alocar mais programas, nesses espaços e isso diminui o problema da fragmentação.

First-fit: neste caso, o sistema operacional escolhe a primeira partição que estiver livre e tiver o tamanho suficiente para o programa. Esta estratégia é mais rápida que as duas anteriores e consome uma quantidade menor de recursos do sistema.

As partições variáveis são implementadas por meio de listas encadeadas.

1.4 REALOCAÇÃO

Como já vimos anteriormente, no caso dos sistemas multiprogramáveis a memória principal é compartilhada com diversos processos, quando um processo sai da memória principal ao retornar a ela, pode ser alocado em uma posição diferente da inicial, assim é necessária a realocação.

Uma forma de resolver a realocação é utilizar registradores de base e limite, ou seja, quando um processo é carregado, o sistema operacional ajusta o valor do registrador de base de acordo com a disponibilidade da memória principal. Este método permite que um processo que já está em execução seja movido na memória.

1.5 MEMÓRIA VIRTUAL

1.5.1 PAGINAÇÃO

Quando utilizamos partições variáveis, temos maior desperdício de memória devido à fragmentação externa. Para que possamos ter um melhor aproveitamento da memória, precisamos que um programa tenha a possibilidade de ocupar áreas não contíguas da memória, eliminando a fragmentação externa. Para fazer esse processo utilizamos a técnica de paginação.

Segundo Machado e Maia (2017, p. 163):

A MEMÓRIA VIRTUAL POR PAGINAÇÃO É A TÉCNICA DE GERÊNCIA DE MEMÓRIA EM QUE O ESPAÇO DE ENDEREÇAMENTO VIRTUAL E O ESPAÇO DE ENDEREÇAMENTO REAL SÃO DIVIDIDOS EM BLOCOS DE MESMO TAMANHO CHAMADOS PÁGINAS. AS PÁGINAS NO ESPAÇO VIRTUAL SÃO DENOMINADAS PÁGINAS VIRTUAIS, ENQUANTO AS PÁGINAS NO ESPAÇO REAL SÃO CHAMADAS DE PÁGINAS REAIS OU FRAMES.

Ao executar um programa o sistema operacional irá transferir as páginas virtuais da memória secundária para a principal. Para que ocorra o carregamento de uma página na memória, existem duas estratégias que são: paginação antecipada e por demanda.

No caso da paginação antecipada, o sistema operacional irá carregar na memória principal, tanto a página referenciada quanto outras páginas que talvez não sejam utilizadas durante seu processamento, porém ao utilizar esta estratégia, podemos ter uma economia de tempo, mas ao mesmo tempo se as páginas que foram carregadas não forem utilizadas, ocorrerá o contrário, a perda de tempo e o uso desnecessário de memória principal. Esta estratégia pode ser utilizada quando ocorrer a criação de um processo ou ainda quando ocorre um *page fault*. O *page fault* ocorre quando um processo referencia um endereço virtual e a página não está na memória, neste caso o processo passa do estado de execução para o estado de espera até a página ser transferida da memória secundária para a principal.

No caso da paginação por demanda, o sistema operacional irá carregar as páginas da memória secundária para a primária, somente quando estas forem referenciadas.

A política de alocação de páginas permite realizar a alocação fixa e a variável. No caso da alocação fixa, cada um dos processos terá um número máximo de frames para serem utilizados durante o processo de execução de um programa.

Um dos problemas no gerenciamento de memória virtual por paginação é a decisão de quais páginas devem ser liberadas, para disponibilizar frames para que novas páginas sejam alocadas. Para realizar esse processo, podemos utilizar algoritmos de substituição de páginas.

Vamos conhecer alguns desses algoritmos, citados por Machado e Maia (2017):

- a) Ótimo: neste algoritmo será selecionado uma página que não será referenciada no futuro ou uma página que levará maior intervalo de tempo para ser utilizada novamente. Este algoritmo garante menores taxas de paginação, porém, é impossível ser implementado já que o sistema operacional não pode prever o comportamento futuro das aplicações, por isso é usado apenas como modelo comparativo de análise de outros algoritmos;
- b) Aleatório: este algoritmo não utiliza um critério de seleção, sendo assim, todas as páginas alocadas na memória principal possuem a mesma chance de serem selecionadas. É pouco usado, mesmo consumindo poucos recursos de sistema, já que possui baixa eficiência;
- c) FIFO (*First In - First Out*): este algoritmo seleciona a página que está há mais tempo na memória principal, aquela que foi utilizada primeiramente. Desta forma, ele utiliza uma estrutura de fila, pois as páginas mais antigas estão no início da fila e as mais recentes no final. Este algoritmo raramente é utilizado sem outro mecanismo que diminua o problema da seleção de páginas antigas que são referenciadas constantemente;
- d) LFU (Least-Frequently-Used): este algoritmo seleciona a página menos referenciada, por meio de um contador com o número de referências para cada página na memória principal. A que tiver menor número de referências será selecionada. Mas, neste caso, pode acontecer das páginas que estão há pouco tempo na memória principal serem selecionadas, já que os contadores terão um número menor de referências ou uma página muito usada no passado pode não ser mais referenciada no futuro, tendo um contador com um número elevado de referências. Normalmente não é utilizado, mas serve como base para outros algoritmos;

- e) LRU (*Least-Recently-Used*): este algoritmo seleciona a página que está há mais tempo sem ser referenciada. Esse algoritmo precisa que cada página tenha associado o momento do último acesso, que deve ser atualizado a cada referência a um frame, assim ele irá buscar um frame que está há mais tempo sem ser referenciado;
- f) NRU (*Not-Recently-Used*): é semelhante ao LRU, mas possui menor sofisticação. É necessário um bit adicional, conhecido como bit de referência que indica se a página foi utilizada recentemente e está presente em cada entrada da tabela de páginas. Ao carregar a página o bit é alterado indicando que a página foi referenciada.

1.5.2 SEGMENTAÇÃO

Segmentação é outra técnica usada para o gerenciamento de memória. Neste caso, o espaço de endereçamento virtual é dividido em blocos que possuem tamanhos diferentes. A esses blocos damos o nome de segmentos. Um programa é dividido de forma lógica em sub-rotinas e estrutura de dados e estas são alocadas nos segmentos da memória principal.

Comparando a técnica de paginação com a técnica de segmentação: na paginação o programa é dividido em páginas de tamanho fixo sem a preocupação com a sua estrutura e na segmentação temos uma relação entre a lógica do programa e sua alocação na memória principal. Quem define os segmentos normalmente é o compilador, a partir do código-fonte do programa, assim cada segmento pode representar um procedimento, função, vetor ou pilha. O espaço de endereçamento virtual possui um número máximo de segmentos que podem existir e cada segmento pode variar de tamanho dentro de um determinado limite (MACHADO; MAIA, 2017).

A vantagem da técnica de segmentação em relação à paginação é a facilidade em lidar com estruturas de dados dinâmicas. Já em relação aos problemas apresentados, na paginação ocorre a fragmentação interna e na segmentação a fragmentação externa.

Outra vantagem da segmentação é o fato da facilidade para compartilhar memória, sendo que cada segmento representa uma parte de um programa que pode ou não ser compartilhado.

Machado e Maia (2017) apresentam outra possibilidade que inclui a junção das duas técnicas, a memória virtual por segmentação com paginação, onde o espaço de endereçamento é dividido em segmentos e cada segmento dividido em páginas, assim aproveita-se as vantagens das duas técnicas.

TABELA 1 - PAGINAÇÃO X SEGMENTAÇÃO

Característica	Paginação	Segmentação
Tamanho dos blocos de memória	Iguais	Diferentes
Proteção	Complexa	Mais simples
Compartilhamento	Complexo	Mais simples
Estruturas de dados dinâmicas	Complexo	Mais simples
Fragmentação interna	Pode existir	Não existe
Fragmentação externa	Não existe	Pode existir
Programação modular	Dispensável	Indispensável
Alteração do programa	Mais trabalhosa	Mais simples

Fonte: Machado e Maia (2017).

Na tabela anterior podemos visualizar um comparativo entre as características da técnica de paginação e segmentação.

1.6 SWAPPING

Quando temos situações onde não é possível que todos os processos sejam alocados simultaneamente na memória, podemos utilizar um mecanismo chamado de swapping. Nesta técnica, existe uma área em disco (memória secundária) reservada pela gerência de memória e quando necessário um processo é copiado para o disco.

Quando necessário, um processo tem sua execução suspensa, ele sai da fila do processador e é colocado em uma fila de processos suspensos, assim ele sofreu o que chamamos de *swap-out* e mais tarde ele será copiado novamente para a memória, sofrendo um *swap-in*. Neste momento, ele volta para a fila do processador e sua execução é retomada. Desta forma, o sistema operacional pode executar mais processos num mesmo instante na memória. O swapping pode ser usado em partições fixa ou variáveis (OLIVEIRA; CARISSIMI; TOSCANI, 2010).

Um algoritmo para escolha do processo que será retirado da memória principal precisa priorizar o processo com menores chances de ser escalonado, pois desta forma se evita realizar swapping que não seja necessário em um determinado momento.

De acordo com Machado e Maia (2017, p. 155), “com a evolução dos sistemas operacionais, novos esquemas de gerência de memória passaram a incorporar a técnica de swapping, como a gerência de memória virtual”.

2. GERENCIAMENTO DE SISTEMAS DE ARQUIVOS

Para compreendermos como funciona o gerenciamento de sistemas de arquivos, é necessário conhecermos os conceitos de arquivos e diretórios.

Os arquivos contêm dados, como por exemplo, um arquivo de texto que possui dados digitados por um determinado usuário. Arquivos podem ser um programa executável, um texto, uma planilha, uma fotografia, uma imagem, um vídeo, entre diversos outros tipos de arquivos.

Já os diretórios são um local onde guardamos arquivos, ou seja, são conjuntos de referências a arquivos. Os diretórios permitem a organização dos arquivos no computador.

2.1 ARQUIVOS

Os arquivos são os locais onde temos dados armazenados e estes possuem uma identificação “nome”, o tipo de conteúdo, como por exemplo, se é um texto ou uma planilha, data e horário de modificação e último acesso, dados do usuário que o criou, lista de usuários que podem ou não acessar este arquivo.

Algumas operações que podem ser realizadas com arquivos no sistema operacional são: criação, exclusão, leitura, edição, execução, renomear, entre outras.

O sistema operacional pode controlar o acesso a arquivos, definir o que cada usuário pode fazer com determinados arquivos. Ao acessar o sistema operacional o acesso do usuário é feito por um login e senha e partir dele é possível verificar as permissões e restrições deste usuário.

Em relação aos nomes de arquivo, Alves (2014, p. 73), afirma que

EXISTEM SISTEMAS QUE FAZEM DISTINÇÃO ENTRE LETRAS MAIÚSCULAS E MINÚSCULAS, COMO É O CASO DO LINUX. NELE, O ARQUIVO DE NOME MINHAAGENDA.DAT DIFERE DE OUTRO COM O NOME MINHAAGENDA.DAT, OU SEJA, SÃO DOIS ARQUIVOS DISTINTOS. NO MS-DOS, AMBOS OS NOMES SERIAM CONSIDERADOS COMO UM ÚNICO ARQUIVO. EM ALGUNS SISTEMAS OPERACIONAIS (PRINCIPALMENTE OS MAIS ANTIGOS, COMO O MS-DOS), É PERMITIDA A INCLUSÃO DE UM CONJUNTO DE CARACTERES (NORMALMENTE TRÊS) QUE REPRESENTA A EXTENSÃO DO ARQUIVO. ESSA EXTENSÃO VEM LOGO APÓS O NOME DO ARQUIVO E É SEPARADA DESSE POR UM PONTO (.). EMBORA NÃO SEJA OBRIGATÓRIO INCLUI-LA, A EXTENSÃO PODE SERVIR PARA AUXILIAR NA IDENTIFICAÇÃO DO TIPO DE ARQUIVO. POR EXEMPLO, NO NOME DE ARQUIVO “CONTRATO.DOC”, A EXTENSÃO É DADA PELA EXPRESSÃO .DOC, QUE REPRESENTA UM ARQUIVO DE DOCUMENTO DO WORD. EM LINUX NÃO EXISTE O CONCEITO DE EXTENSÃO DE ARQUIVO, MAS PODEMOS ADICIONAR O PONTO SEGUIDO DAS LETRAS QUE REPRESENTAM A EXTENSÃO, EMBORA ELE A TRATE COMO SE FOSSE PARTE LEGÍTIMA DO NOME DO ARQUIVO. NO WINDOWS, A PARTIR DA VERSÃO 95, A EXTENSÃO PASSOU A TER MAIS UMA FUNÇÃO DE VÍNCULO DO ARQUIVO COM O PROGRAMA QUE O MANIPULA, EMBORA SEJA POSSÍVEL AO USUÁRIO ASSOCIAR UM PROGRAMA A UM TIPO DE ARQUIVO SEM LEVAR EM CONTA A EXTENSÃO DELE. ESSA ASSOCIAÇÃO TEM COMO FINALIDADE PERMITIR QUE O PROGRAMA CORRESPONDENTE AO ARQUIVO SEJA EXECUTADO AUTOMATICAMENTE QUANDO O USUÁRIO CLICAR DUAS VEZES NELE. POR EXEMPLO, SE FOR DADO UM DUPLO CLIQUE EM UM ARQUIVO COM A EXTENSÃO .DOC, O WORD É ABERTO COM O ARQUIVO JÁ CARREGADO.

A forma como os arquivos são armazenados, pode ser diferente, de acordo com o tipo de informação que contém. A organização do arquivo pode ser definida pela aplicação ou pode ser uma estrutura que é suportada pelo sistema operacional. Cada sistema operacional pode utilizar diferentes organizações de arquivos.

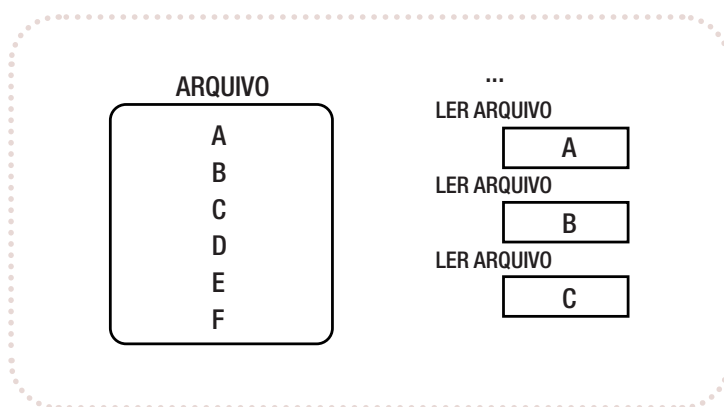
Segundo Machado e Maia (2017), os tipos de organizações de arquivos que são mais conhecidas e utilizadas são: sequencial, relativa e indexada. De acordo com a forma como os arquivos estão organizados, sua recuperação também pode ocorrer de maneiras diferentes.

Por exemplo, nos antigos sistemas operacionais que usavam a gravação de arquivos em fitas magnéticas a gravação de novos registros era realizada de forma sequencial. Com o surgimento de discos magnéticos foi possível a leitura e gravação de registros diretamente na sua posição.

Para um sistema operacional um arquivo corresponde a uma sequência de *bytes*, a não ser os arquivos executáveis que possuem uma estrutura interna definida pelo próprio sistema operacional.

Para acesso ao conteúdo de um arquivo, podemos usar o acesso sequencial, onde o arquivo é lido na sequência, como mostra a figura a seguir.

FIGURA 3 - ACESSO SEQUENCIAL



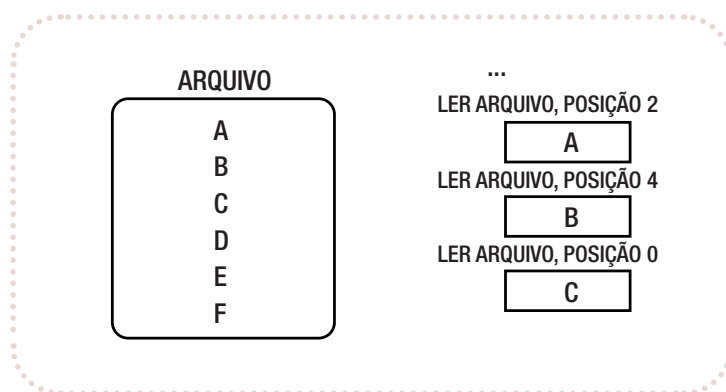
Fonte: Oliveira, Carissimi e Toscani (2010, p. 212).

Outra forma de acesso é o acesso relativo, que permite acessar um registro sem precisar ler todo o arquivo sequencialmente. Oliveira, Carissimi e Toscani (2010, p. 212), exemplificam este tipo de acesso:

[...] CONSIDERE UM ARQUIVO QUE CONTÉM UM CADASTRO DE FUNCIONÁRIOS. O CONTEÚDO DO ARQUIVO É ORGANIZADO EM REGISTROS, E CADA REGISTRO CONTÉM OS DADOS RELATIVOS A UM FUNCIONÁRIO EM PARTICULAR. OS FUNCIONÁRIOS SÃO NUMERADOS DE TAL FORMA QUE OS DADOS RELATIVOS AO FUNCIONÁRIO NÚMERO 1 É ARMAZENADO NO REGISTRO NÚMERO 1, OS DO FUNCIONÁRIO NÚMERO 2 NO REGISTRO 2, E ASSIM POR DIANTE. SUPONHA QUE UM PROGRAMA DESEJA ACESSAR O REGISTRO DO FUNCIONÁRIO NÚMERO 100. ELE ACABA DE RECEBER UMA PROMOÇÃO E ESSE FATO DEVE SER INCLUÍDO EM SEU REGISTRO. O ACESSO SEQUENCIAL OBRIGARIA O PROGRAMA A LER OS REGISTROS DE 99 FUNCIONÁRIOS, MESMO QUANDO ISSO NÃO É NECESSÁRIO.

No exemplo citado, é usado o acesso relativo, pois o programa inclui qual posição deve ser lida, conforme ilustra a figura a seguir.

FIGURA 4 - ACESSO RELATIVO



Fonte: Oliveira, Carissimi e Toscani (2010, p. 212).

Mais adiante, vamos falar sobre alguns sistemas operacionais mais específicos, porém vamos compreender um pouco sobre os sistemas de arquivos do Windows e Linux.

Um dos tipos de sistemas de arquivo do Windows é o chamado FAT – File Allocation Table ou Tabela de Alocação de Arquivos, que é similar a um mapa para facilitar a localização de um arquivo em disco para que este possa ser acessado ou manipulado. Depois surgiu o sistema de arquivos chamado NTFS que vem de NT File System, que foi criado a partir do Windows NT. Atualmente é o sistema utilizado pelo Windows tanto de 32 quanto de 64 bits.

No Linux o sistema de arquivos mais conhecido é o ext2 (sistema de arquivo estendido) que possui alto desempenho e suporte a recursos avançados. Ele utiliza grupo de blocos e cada bloco é formado por um conjunto de setores. Na versão ext3 temos um recurso chamado *journaling* que registra as transações efetuadas e permite recuperar o sistema quando necessário.

Após conhecermos sobre os conceitos relacionados a arquivos, vamos compreender como funcionam os diretórios, pois são nos diretórios que os arquivos são armazenados.

2.2 DIRETÓRIOS

Para organizarmos nossos papéis e materiais no mundo real, temos armários, gavetas, pastas, entre outras opções. Em um sistema computacional também precisamos organizar nossos arquivos, os arquivos de cada programa, arquivos de vídeos, fotos, textos, planilhas, etc. A estrutura de diretórios é um sistema que organiza de forma lógica os arquivos que estão em um disco. Esta estrutura possui entradas associadas aos arquivos e armazena informações sobre ele, como seus atributos (nome, localização física, data de criação, entre outros). No Windows, por exemplo, chamamos também, os diretórios de pastas.

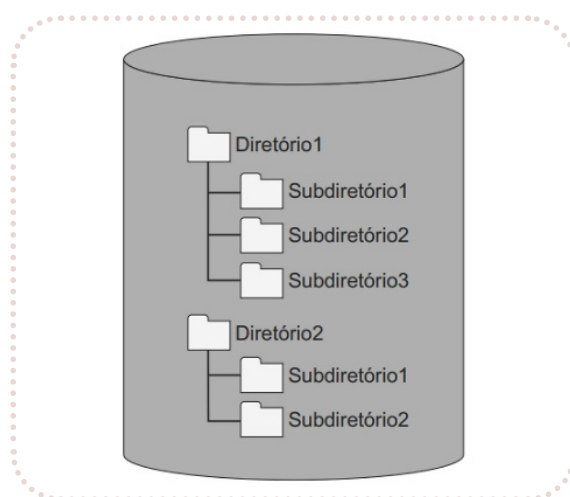
A estrutura de diretórios é hierárquica, podemos criar um diretório dentro do outro, esses são chamados subdiretórios. No Windows, por exemplo, caso um diretório seja apagado, tudo que está dentro dele será apagado também, sejam arquivos ou subdiretórios.

Ao abrir um arquivo o sistema operacional irá buscar sua entrada na estrutura de diretórios e então armazena seus atributos e localização em uma tabela que fica na memória principal com todos os arquivos abertos neste momento. Quando terminamos de usar um arquivo, é importante fechá-lo para que libere espaço na memória, por exemplo, ao terminar de usar um arquivo de texto, você deve fechá-lo.

Quando temos vários usuários utilizando um mesmo computador ou rede, por exemplo, é necessário nos preocuparmos com uma hierarquia geral, uma árvore de diretórios, para que cada usuário possa ter a quantidade de diretórios que precisar.

Na figura a seguir podemos ver um exemplo de estrutura de diretórios que podem ser utilizados nos sistemas operacionais.

FIGURA 5 - EXEMPLO DE ESTRUTURA DE DIRETÓRIOS



Fonte: Alves (2014).

Podemos realizar várias operações com diretórios, assim como fazemos com arquivos. Cada sistema operacional tem uma forma de trabalhar com seus diretórios, mas em geral, podemos fazer operações como criar, excluir, renomear, mover e copiar.

2.3 CACHE DE SISTEMAS DE ARQUIVOS

Em um sistema de arquivos temos a estrutura chamada cache, diferente do cache usado pelo processador.

Oliveira, Carissimi e Toscani (2010, p. 212) afirmam que os caches apresentam um grande aumento no desempenho de um sistema de arquivos, o que tem grande importância porque o uso do disco tende a ser intenso em sistemas operacionais de propósito geral. O cache mantém na memória principal uma determinada quantidade de blocos do disco, assim caso algum desses blocos seja requisitado para leitura ou escrita, ele será encontrado na memória principal e isto evita acessar o disco, o que tornaria o processo mais demorado.

Este tipo de cache usa uma área da memória principal e é controlada pelo próprio sistema operacional.

2.4 GERÊNCIA DO ESPAÇO LIVRE

O sistema operacional gerencia o espaço livre nos discos para determinar onde temos setores livres que podem ser alocados. Existem sistemas operacionais que agrupam setores em blocos físicos, onde o disco é visto como uma sequência de blocos físicos e não de setores.

O espaço em disco pode ser gerenciado através de um mapa de bits, onde o sistema operacional percorre este mapa até achar um bit zero para que possa alocar um bloco físico para um determinado arquivo. Este método permite alocar áreas contíguas no disco.

Outra forma de realizar este gerenciamento é através de uma lista que contenha os números dos blocos físicos livres que é mantida no disco.



LEITURA

Para aprender mais sobre pastas e arquivos no Windows 10, faça a leitura do artigo: **Como habilitar a busca avançada do Windows 10 para o menu Iniciar.**

Disponível em: <https://bit.ly/36xOmFi>. Acesso em 18 jun. 2021.

3. GERENCIAMENTO DE DISPOSITIVOS DE ENTRADA/SAÍDA

O sistema operacional controla os dispositivos de entrada e saída e permite que a interface com eles seja simples. Possuímos diversos dispositivos de entrada, de saída e que podem ser de entrada e saída ao mesmo tempo, como por exemplo, teclado, mouse, monitor, impressora, pendrive, entre diversos outros. Esses dispositivos também podem ser chamados de periféricos e permitem realizar uma interação com o sistema.

Muitas vezes ao instalar um novo dispositivo de entrada ou saída, precisamos instalar um driver ou device driver. O driver tem a finalidade de realizar a comunicação do subsistema de entrada/saída com os controladores. Eles recebem comandos gerais e transformam em comandos específicos, os quais serão executados pelos controladores. Como existe uma relação de dependência entre um driver e o núcleo do sistema, os drivers em geral, são desenvolvidos para cada sistema operacional.

Já os controladores, são responsáveis por manipular os dispositivos de entrada/saída, onde os drivers se comunicam com os dispositivos por meio de um controlador.

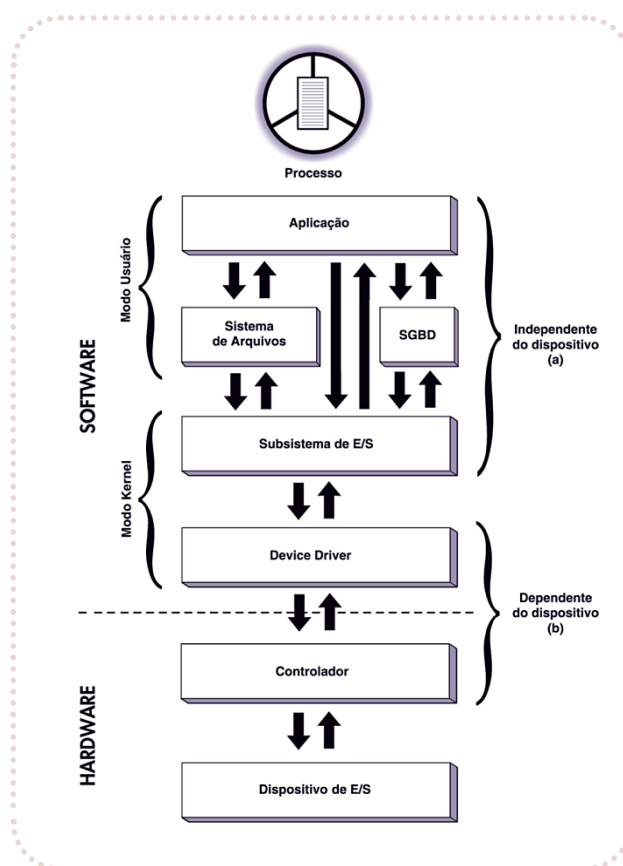
Nos dispositivos de entrada/saída, os dados podem ser transferidos por meio de blocos de informação ou de caracteres utilizando os controladores desses dispositivos. De acordo como os dados são armazenados podemos classificar esses dispositivos em estruturados (armazenam in-

formações em blocos que possuem tamanho fixo) ou não estruturados (enviam ou recebem uma sequência de caracteres sem estar estruturada em formato de bloco).

Os dispositivos de entrada/saída são conectados por meio de uma interface com o computador. Além da interface é necessário outro componente chamado de controlador que permite implementar operações como leitura e escrita de dados, reinicialização, etc. Os dispositivos podem ser tipo serial ou tipo paralelo, onde a interface serial possui apenas uma linha para os dados e a interface paralela possui várias linhas para os dados.

Um subsistema de entrada e saída é composto por um conjunto de rotinas que possibilita a comunicação com qualquer dispositivo conectado ao computador. Esse conjunto de rotinas permite ao usuário realizar operações de entrada e saída sem se preocupar com detalhes do dispositivo que está sendo acessado (MACHADO; MAIA, 2017). Na figura a seguir podemos visualizar diferentes maneiras de uma aplicação interagir com o subsistema de entrada e saída.

FIGURA 6 - ARQUITETURA DE CAMADAS DA GERÊNCIA DE DISPOSITIVOS



Fonte: Machado e Maia (2017, p. 209).

Um dos dispositivos que podemos exemplificar como entrada/saída são os discos rígidos (magnéticos) que ainda são tidos como um dos principais locais de armazenamento de dados e também do sistema operacional.

Segundo Machado e Maia (2017), no caso dos discos magnéticos, o tempo para leitura e gravação de um bloco de dados é calculado em relação a três fatores: tempos de *seek*, latência ro-

tacional e transferência. O tempo de *seek* se refere ao tempo utilizado para o posicionamento do mecanismo de leitura e gravação até o cilindro do bloco no qual se encontra, o tempo de latência rotacional está relacionado ao tempo de espera até que o setor desejado se posicione sob o mecanismo de leitura e gravação. E por último, o tempo de transferência é o tempo necessário para que ocorra a transferência do bloco entre a memória principal e o setor do disco.

Os discos rígidos podem ser divididos no que chamamos de partições, onde cada partição pode ter diferentes tipos de sistema operacional, por exemplo. Os discos rígidos possuem um setor na posição zero (0) que é chamado de **Master Boot Record**, que é lido pela BIOS quando o computador é ligado, antes de iniciar o carregamento do sistema operacional.



REFLITA

Como as interrupções podem ajudar na melhoria do desempenho dos processos executados em um sistema computacional?

A interrupção avisa ao processador quando o evento ocorreu, permitindo dessa forma uma acomodação eficiente para dispositivos mais lentos. Interrupções permitem aos processadores modernos responder a eventos gerados por dispositivos enquanto outro trabalho está sendo realizado.

3.1 INTERRUPÇÕES

No que diz respeito aos dispositivos de entrada/saída, devemos considerar também, o mecanismo de interrupção. Esse tipo de mecanismo ocorre quando existe a geração de um sinal para o processador devido a um evento externo a ele. Quando o processador recebe este sinal ele irá parar por um tempo e executar outra ação, ao terminar, voltará para a rotina que foi interrompida. Em geral, o hardware possui um controlador de interrupções. Mas, as interrupções podem ser ativadas tanto por hardware quanto por software. No caso de interrupções por software, este executa uma instrução específica que foi programada. Ainda pode haver interrupções devido a um erro na execução de um programa, por exemplo.

A utilização de interrupções permite determinar quando um dispositivo de entrada/saída, precisa de atenção do processador. Quando temos um volume grande de dados este procedimento pode ser ineficaz, já que precisa realizar o movimento de dados do controlador para o processador e do processador para a memória.

Para resolver o problema citado acima, pode-se utilizar a técnica de acesso direto a memória ou DMA (**Direct Memory Access**). Essa técnica utiliza o controlador de DMA para realizar a transferência de dados entre o dispositivo de entrada/saída e a memória. Desta forma, evita sobrecarregar o processador, pois as transferências de dados acontecem sem a intervenção da CPU, o que permite maior velocidade nas ações.

Mais um exemplo de periférico que podemos citar, é o teclado o qual é um dos periféricos mais utilizados em computadores e permite a interação entre o usuário e o computador. Basicamente

ele funciona ao pressionarmos uma tecla onde um símbolo é gerado. Este processo de pressionar uma tecla também gera uma interrupção no hardware.

3.2 SOFTWARE DE ENTRADA E SAÍDA

Por meio de software há possibilidade de acessar dispositivos de entrada e saída, poderíamos, por exemplo, acessar discos ou outros meios de armazenamento a partir do momento que um programa tenha capacidade de ler um arquivo nesta mídia.

Um software para entrada e saída de dados, precisa prever o tratamento de erros que deve ser tratado o mais próximo possível do hardware, ou seja, se o controlador verifica um erro de leitura ele deve tentar corrigir. O sistema operacional é encarregado de fazer com que operações baseadas em interrupções pareçam bloqueios para os softwares de usuário.

Os softwares de entrada e saída podem fazer o uso de buffers, quando não podem ser armazenados diretamente em seu destino final. Exemplo: um dispositivo de áudio digital pode ter restrições de tempo real, assim os dados devem ser colocados antecipadamente em um buffer de saída para desvincular a velocidade com que ele é preenchido da velocidade com que ele é esvaziado, assim evita a falta de dados. A utilização de buffers envolve um volume de cópias considerável e frequentemente tem um impacto importante sobre o desempenho das operações de entrada e saída (TANENBAUM; WOODHULL, 2008).

Para Oliveira, Carissimi e Toscani (2010, p. 132), “o subsistema de entrada e saída de um sistema operacional é um software bastante complexo, devido principalmente à diversidade de periféricos encontrados atualmente, os quais ele deve tratar”.

Esse subsistema procura padronizar as rotinas de acesso aos dispositivos, para minimizar o número de rotinas de entrada e saída. Essa característica permite a inclusão de novos dispositivos sem alterar a interface de programação com o usuário.



LEITURA

Faça a leitura do artigo: **Como funcionam os dispositivos de entrada e saída**, e saiba como funcionam os dispositivos de entrada e saída e como se dá sua comunicação com o sistema operacional.

Disponível em: <https://bit.ly/3z0u6bp>. Acesso em 18 jun. 2021

CONSIDERAÇÕES FINAIS

Nesta Unidade, foi possível compreendermos o funcionamento do gerenciamento de memória, gerenciamento de arquivos e gerenciamento de dispositivos.

UNIDADE

03

SISTEMAS OPERACIONAIS DE MERCADO E REDUZIDOS



OBJETIVOS DE APRENDIZAGEM

- » Conhecer os principais sistemas operacionais disponíveis no mercado;
- » Conhecer os principais sistemas operacionais voltados a dispositivos móveis.



VÍDEOS DA UNIDADE



<https://bit.ly/2Vah24D>



<https://bit.ly/3kSyFRo>



<https://bit.ly/3y3UP6Z>

INTRODUÇÃO

Nesta Unidade vamos conhecer os principais sistemas operacionais utilizados em computadores pessoais, mainframes e sistemas operacionais para dispositivos móveis.

Primeiro vamos abordar uma visão geral sobre os sistemas operacionais, depois vamos conhecer o sistema operacional Windows, tido como o sistema operacional mais utilizado em computadores pessoais, tanto para uso doméstico quanto nas organizações.

Depois vamos falar sobre o Linux, que é um sistema operacional de código aberto, sobre o Mac OSX, o sistema operacional dos computadores da Apple e o z/OS, sistema operacional da IBM, utilizado em mainframes, computadores de grande porte.

Na sequência, vamos falar sobre os sistemas operacionais voltados a dispositivos móveis, abordando o Android e o iOS.

É de grande importância na área de tecnologia da informação, o conhecimento sobre os sistemas operacionais, pois eles são necessários em todas as atividades que envolvem equipamentos eletrônicos, sendo hoje em dia, utilizados não só em computadores e smartphones, mas também, em diversos outros equipamentos, como smart TVs e equipamentos eletrônicos diversos.

Também é necessário o seu conhecimento para que possamos desenvolver aplicativos, pois cada plataforma tem suas especificidades durante o desenvolvimento.

1. SISTEMAS OPERACIONAIS DE MERCADO

Antes de falarmos um pouco sobre os principais sistemas operacionais que estão no mercado, vamos pensar porque é difícil projetar um sistema operacional.

Segundo Tanenbaum e Bos (2016), a lei de Moore diz que o hardware de computadores é melhorado por um fator de 100 a cada década, porém não existe nenhuma lei sobre o sistema operacional ser melhorado. Os mesmos autores apresentam oito questões que tornam o projeto de sistemas operacionais complexos:

- » Primeira: os sistemas operacionais são programas extensos demais, sendo assim não é possível escrever um sistema operacional às pressas, em pouco tempo;
- » Segunda: os sistemas operacionais têm de lidar com a concorrência, ou seja, são múltiplos usuários e dispositivos de entrada e saída, todos ativos de uma só vez. Esse gerenciamento é muito mais difícil do que o gerenciamento de uma única atividade sequencial;
- » Terceira: os sistemas operacionais lidam com usuários que querem interferir no funcionamento do sistema ou fazer coisas proibidas, por isso, ele precisa tomar medidas para evitar que esses usuários se comportem inadequadamente;

- » Quarta: desconsiderando o fato de que nem todos os usuários confiam uns nos outros, muitos usuários querem compartilhar informações e recursos com outros usuários e aí, o sistema precisa tornar isso possível, de modo que os usuários mal-intencionados não possam interferir;
- » Quinta: os sistemas operacionais vivem por um longo tempo. O UNIX, por exemplo, vem sendo usado por aproximadamente 40 anos, o Windows por volta de 30 anos, assim é preciso pensar em como o hardware e as aplicações poderão mudar no futuro distante e como eles devem se preparar para isso;
- » Sexta: os projetistas de sistemas operacionais realmente não têm uma boa ideia de como seus sistemas serão usados, então eles precisam desenvolvê-los visando a uma considerável generalidade;
- » Sétima: os sistemas modernos geralmente são projetados para serem portáteis, o que significa que têm de executar em múltiplas plataformas de hardware. Precisam suportar diversos dispositivos de entrada e saída e todos são projetados independentemente, sem qualquer relação uns com os outros;
- » Oitava: a frequente necessidade de manter a compatibilidade com algum sistema operacional anterior.

1.1 WINDOWS

O sistema operacional Windows, da empresa Microsoft, possui interface gráfica e permite trabalharmos com vários programas de forma simultânea, pois é um sistema multitarefa. Atualmente mais de 80% dos computadores pessoais utilizam alguma versão do sistema operacional Windows. Inicialmente o Windows não foi concebido como sistema operacional, ele era uma interface gráfica para o sistema operacional MS-DOS ou outro sistema compatível, no início dos anos 1990.

Ainda nos anos 1990, surgiram diversas versões do Windows, que passou a ter as características de sistema operacional. As versões foram surgindo com melhorias e resolução de problemas apresentados em versões anteriores. Atualmente a versão mais utilizada é o Windows 10, que foi lançada oficialmente em 2015.

Na tabela na sequência, podemos observar as principais versões do Windows, antes da versão Windows 10, com suas datas de lançamento.



TABELA 1 - PRINCIPAIS LANÇAMENTOS NA HISTÓRIA DOS SISTEMAS OPERACIONAIS MICROSOFT PARA PCS DESKTOP

Ano	MS-DOS	Windows baseado no MS-DOS	Windows baseado em NT	Windows moderno	Observações
1981	1.0				Distribuição inicial para IBM PC
1983	2.0				Suporte para PC/XT
1984	3.0				Suporte para PC/AT
1990		3.0			Dez milhões de cópias em 2 anos
1991	5.0				Incluído gerenciamento de memória
1992		3.1			Funcionava somente em 286 ou superior
1993			NT 3.1		
1995	7.0	95			MS-DOS embutido no Win 95
1996			NT 4.0		
1998		98			
2000	8.0	Me	2000		Win Me era inferior ao Win 98
2001			XP		Substituiu o Win 98
2006			Vista		Vista não conseguiu suplantar o XP
2009			7		Melhoria significativa sobre o Vista
2012				8	Primeira versão moderna
2013				8.1	Microsoft passou para lançamentos rápidos

Fonte: Tanenbaum e Bos (2016).

O Windows permite que mais de uma pessoa, ao acessar um mesmo computador, por exemplo, em casa, utilize usuários diferentes. Isso permite que os arquivos de um usuário não sejam visualizados ou acessados por outros usuários. Mas, é possível criar uma pasta e compartilhar com todos do grupo. Por outro lado, se tivermos vários computadores em uma rede e quisermos usar um recurso único, como uma única impressora, também é possível, por meio do compartilhamento.

Em relação à arquitetura do sistema operacional, o Windows é inspirado principalmente no princípio de micronúcleo, desta forma as funcionalidades são gerenciadas por um único componente do Windows. Mas, não é totalmente orientado a esta filosofia, porque módulos fora do micronúcleo executam operações em modo protegido. Além disso, ele é organizado em camadas, sendo os módulos dispostos uns sobre os outros. Este sistema é orientado a objetos, onde seus recursos são implementados desta forma e manipulador por meio de métodos que estão associados aos objetos.

1.1.1 GERENCIAMENTO DE ENTRADA E SAÍDA

No Windows, o sistema de gerenciamento de entrada e saída, fica responsável pelo acesso ao sistema de arquivos, gerencia o cache de dados e os drivers de dispositivos e rede.

Quando existe uma requisição de entrada e saída, estas são convertidas pela gerência de entrada e saída do sistema operacional em um formato chamado IRP – *I/O Request Packet*. Depois, este IRP, é direcionado ao driver responsável pela operação que será realizada e ao finalizar esta operação, o driver de dispositivo sinaliza a gerência, desta forma oferece suporte tanto para operações síncronas quanto assíncronas.

As operações de entrada e saída são realizadas pelos drivers de cada dispositivo, os quais iniciam um objeto de driver que possui os endereços dos procedimentos. Os dispositivos de entrada e saída são representados pelos objetos de dispositivos que são criados a partir da descrição da configuração do sistema ou pelo gerenciador *plug and play*, conforme ele descobre dispositivos quando vai enumerando os barramentos do sistema. Esses dispositivos são empilhados e os pacotes de solicitação de entrada e saída são repassados para a pilha de dispositivos. Os drivers, geralmente enfileiram as solicitações para processamento futuro e retorno a quem os chamou (TANENBAUM; BOS, 2016).

Um dispositivo *plug and play*, se refere à capacidade do computador reconhecer e configurar automaticamente um dispositivo de entrada e saída, como por exemplo, um mouse, um teclado, entre outros.

1.1.2 PROCESSOS, THREADS E SEÇÕES

Para compreendermos os conceitos de processos, threads e seções utilizadas no Windows, vejamos: os processos têm espaços de endereçamento virtual, já os threads são a unidade de execução e as seções representam objetos na memória, como por exemplo, arquivos, que podem ser mapeados nos espaços de endereçamento dos processos.

Os processos no Windows ocorrem por meio dos objetos “processo” e “thread”, onde o primeiro corresponde a recursos do sistema (memória, arquivos, entre outros) e o segundo está relacionado a uma unidade de trabalho executada de forma sequencial, podendo ser interrompida. Um processo corresponde a criar um objeto do tipo processo e nele diversos atributos são inicializados para esse novo processo. Já a unidade de escalonamento é o conceito de thread. A cada processo pelo menos uma thread está associada a ele. E cada uma delas, pode criar outras threads. Isso permite a execução concorrente dos processos, além de possibilitar uma concorrência entre threads que pertencem a um mesmo processo (OLIVEIR; CARISSIMI; TOSCANI, 2010).

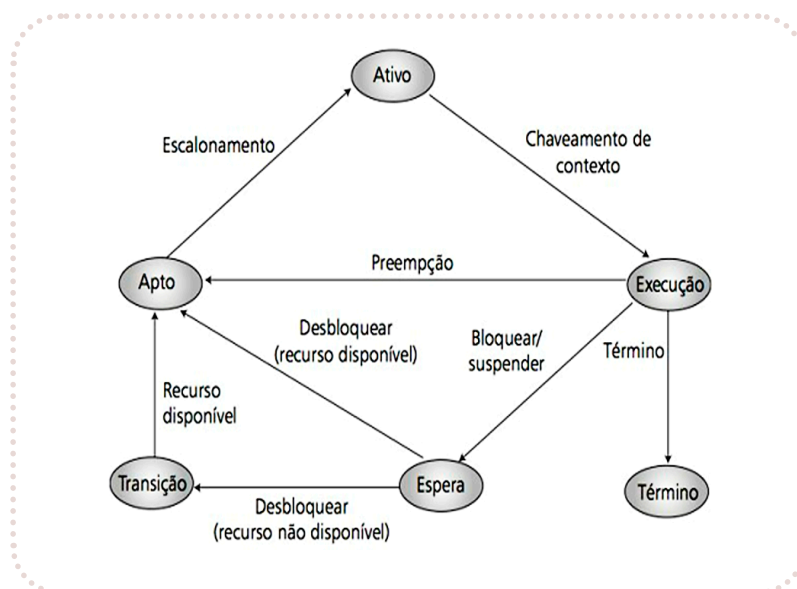
Ainda, segundo Oliveira, Carissimi e Toscani (2010), uma thread pode estar em um dos seis estados, que são:

- » Apto (*ready*): neste estado estão as threads aptas a executar, ou seja, as threads que o escalonador considera para selecionar a próxima a ser executada. Depois de selecionada, a thread passa para o estado a seguir;
- » Ativa (*standby*): este é um estado intermediário, onde a thread selecionada pelo escalonador aguarda o chaveamento de contexto para entrar efetivamente em execução;
- » Em execução (*running*): neste momento uma thread está ocupando o processador e será executada até que ela seja preemptada por uma thread de prioridade mais alta, esgote a sua fatia de tempo, realize uma operação bloqueante, ou seja finalizada;
- » Espera (*waiting*): ela estará neste estado sempre que for bloqueada pela espera da ocorrência de um evento ou realizar uma primitiva de sincronização, ou ainda, quando um subsistema ordena a suspensão da thread.

- » Transição (*transition*): é o estado em que uma thread está apta a ser executada, mas os recursos de sistema necessários a sua execução, ainda não estão disponíveis.
- » Término (*terminated*): é o estado quando chega ao seu final ou é finalizada por outra thread, ou ainda, quando o processo ao qual está associada encerra.

Na imagem a seguir, podemos observar um diagrama com os estados de threads no sistema operacional Windows.

FIGURA 1 - DIAGRAMA DE ESTADOS DE THREADS NO WINDOWS



Fonte: Oliveira, Carissimi e Toscani (2010, p. 280).

1.1.3 SISTEMA DE ARQUIVOS

Um sistema de arquivos, nada mais é do que um conjunto de estruturas lógicas que permitem que o sistema operacional acesse e controle dados que são gravados no disco.

O sistema de arquivos utilizado pelo Windows é o NTFS que vem da sigla NT File System, mas também possui suporte a sistemas como: FAT, FAT32, HPFS, CDFS e UDF (*Universal Disk Format*).

O sistema NTFS, atende alguns pontos críticos para aplicações, no caso de ambientes corporativos, tais como: facilidade para recuperar dados, tolerância a falhas, segurança, suporte a discos de grande capacidade, fluxo de dados múltiplos, facilidade de indexação e ainda, suporte para sistemas POSIX, suporta configuração de permissões e também criptografia.

Com esse sistema de arquivo é possível configurar permissões para cada tipo de arquivo, evitando que usuários não autorizados acessem o que não deveriam. Também, mantém um registro de todas as alterações realizadas em um arquivo.

Esse tipo de sistema de arquivos é baseado em uma tabela mestre de arquivos, a qual possui um registro para cada arquivo ou diretório. Os arquivos possuem diversos atributos.

1.2 LINUX

Na década de 1960 o sistema Multics foi projetado pelo MIT (*Massachusetts Institute of Technology*), pela GE (General Eletric) e pelos laboratórios Bell (Bell Labs) e AT&T (*American Telephone and Telegraph*), e o objetivo era que este sistema utilizasse tempo compartilhado. Após a Bell Labs se retirar do projeto algum tempo depois, Ken Thompson, continuou suas pesquisas com a ideia de criar algo menor com a base do sistema Multics. Deste trabalho, começou a ser criado o sistema Unix. Já em 1973, Dennis Ritchie, reescreveu o Unix em uma linguagem de alto nível, chamada C que também foi criada por ele. Entre os anos de 1977 e 1981, a AT&T alterou o Unix e lançou o System III. Em 1983, com diversas alterações foi lançado o Unix System IV.

A denominação Linux, surgiu da mistura de Linus + Unix, onde o nome Linus é o do seu criador Linus Torvalds, que iniciou seu desenvolvimento em 1991. Mesmo com a falta de diversas características implementadas em sistemas operacionais estabelecidos como o Unix, muitos desenvolvedores deram apoio àquele sistema que era disponível de forma gratuita.

O termo Linux, começou a ser utilizado como uma referência aos sistemas operacionais que eram desenvolvidos com o kernel do Linux, assim tais sistemas, são chamados de distribuições Linux ou ainda de sistemas operacionais baseados em Linux. Os mais populares são: Ubuntu, Fedora, SuSE e Debia. Em 1994, o Linux possuía diversas características que são encontradas em sistemas operacionais, como multiprogramação, memória virtual, entre outros.

A partir do desenvolvimento e melhorias o projeto começou a adotar a numeração de versões. Durante o processo de desenvolvimento, vão sendo introduzidas novas características que nem sempre foram testadas, assim podem não ser confiáveis neste momento. A partir do momento que um núcleo não contém mais nenhum erro conhecido, fica apto para liberação.

No final dos anos 1990 o Linux já possuía diversos recursos, porém ainda era ignorado por muitos usuários, pelo fato de ser difícil de usar e instalar.

Em 2001 a versão 2.4 foi liberada e nela vários subsistemas de núcleo foram alterados e até mesmo reescritos para suportar máquinas mais novas ou ainda, usar as já existentes de forma mais eficiente. A versão 2.6 já era voltada a escalabilidade, conformidade aos padrões e também, as modificações de subsistemas de núcleo permitindo maior desempenho.

Um sistema Linux possui interfaces e aplicações de usuários, como por exemplo, um interpretador de comando shell e utiliza a abordagem em camadas como no UNIX. Ele é um sistema multiusuário e possui controle de restrições, conforme os privilégios de determinado usuário.

Quando o computador é inicializado a BIOS executa um autoteste POST (*Power-On-Self-Test*), para detectar e inicializar os dispositivos. Depois o primeiro setor do disco é lido e executado, carregando um programa chamado boot do dispositivo de boot.

1.2.1 INTERFACE

Atualmente os sistemas Linux possuem uma interface gráfica, mas diversos desenvolvedores e também usuários, preferem a interface de linha de comando que é chamada de interpretador de comandos ou shell, por ser mais rápida, eficiente, possuir facilidade de extensão, entre outros.

Ao utilizar o shell, este inicializa a si mesmo, iniciando um percentual o sinal de dólar e espera que o usuário digite a linha de comando. Para os usuários que gostam de utilizar uma interface gráfica, o Linux possui diversas GUIs (Interface Gráfica de Usuário) disponíveis gratuitamente.

1.2.2 SISTEMAS DE ARQUIVOS

No Linux, o sistema de arquivos possui algumas derivações do Multics, MS-Dos, Windows, entre outros sistemas operacionais. Porém, também temos aquelas que são baseadas exclusivamente no sistema Unix. Os arquivos do Linux são uma sequência de zeros (0) ou mais bytes e os nomes de arquivos são limitados a 255 caracteres.

Os arquivos podem ser agrupados em diretórios e os diretórios são armazenados como arquivos. Os diretórios podem conter subdiretórios e o diretório raiz é chamado de /, sendo que este caractere (/) pode ser usado também, para separar os nomes de diretórios.

Na tabela a seguir, podemos ver a estrutura de diretórios do Linux.

TABELA 2 – ESTRUTURA DE DIRETÓRIOS DO LINUX

Diretório	Descrição
bin	Contém arquivos binários (executáveis e bibliotecas dinâmicas) essenciais ao sistema operacional.
boot	Arquivos necessários à inicialização do sistema e o Kernel propriamente dito.
dev	Dispositivos de entrada e saída, como portas seriais, USB, leitor de CD/DVD etc.
etc	Arquivos de configuração e inicialização do sistema.
home	Diretório local do usuário, contendo arquivos pessoais de cada um.
lib	Bibliotecas e módulos compartilhados entre os programas.
mount	Montagem de dispositivos e sistema de arquivos.
opt	Diretório para instalação de programas não oficiais.
proc	Diretório virtual onde rodam os processos ativos.
root	Diretório do superusuário.
temp	Diretório para armazenamento de arquivos temporários gerados por programas aplicativos.
usr	Diretório para arquivos de usuário nativos do sistema.
var	Diretório destinado a conter arquivos de log.

Fonte: Alves (2014, p. 120).

1.2.3 DISPOSITIVOS DE ENTRADA E SAÍDA

No Linux praticamente todos os dispositivos de entrada e saída parecem arquivos e são acessados como se fossem arquivos, utilizando as mesmas chamadas de sistema de leitura e escrita (read/write) utilizadas para ter acesso a outros tipos de arquivos. Os computadores que utilizam o Linux possuem dispositivos de entrada e saída como discos, impressoras, etc.

O sistema operacional integra estes dispositivos em um sistema de arquivos que é denominado de “arquivos especiais”, os quais são divididos em bloco e caractere. Arquivos de bloco

consistem em uma sequência de blocos numerados e cada um pode ser individualmente endereçado e acessado. Arquivos de caracteres são usados para dispositivos que realizam entrada ou saída de um fluxo de caracteres. Normalmente cada dispositivo de entrada e saída possui um arquivo especial associado a ele.

Segundo Tanenbaum e Bos (2016), quando um usuário acessa algum arquivo especial, o sistema de arquivos vai determinar os números de dispositivos maiores e menores pertencendo a ele e se ele é um arquivo de bloco especial ou de caracteres. O número do dispositivo maior será usado para indexar em uma ou mais tabelas de espalhamento internas que contém estruturas de dados para dispositivos de bloco e de caracteres. A estrutura localizada irá conter os ponteiros para os procedimentos para realizar uma chamada que irá abrir, ler e escrever no dispositivo. Já o número do dispositivo menor é passado como um parâmetro.

1.3 DIFERENÇAS ENTRE WINDOWS E LINUX

Neste tópico vamos comentar sobre algumas diferenças entre os sistemas operacionais Windows e Linux.

Uma das diferenças é que o Windows é um sistema operacional comercial pago e o Linux é um sistema gratuito. Outra questão, é que o Windows utiliza um único banco de dados para configurar seu registro e o Linux não possui este recurso, pois não tem registro e seus arquivos de configuração ficam no diretório raiz.

No Windows, o controle de acesso de usuários é feito por meio de login como administrador do sistema, mas isso abre espaço para que sejam infectados por vírus ou ainda por malwares, já que muitos programas são executados pelos usuários, que nem sempre têm conhecimento suficiente sobre esta questão. As últimas versões do Windows já possuem mais controle de acesso. Já no Linux o administrador ou “root” só utiliza esse acesso se for necessário, pois o login em geral, é feito como usuário comum e não como administrador.



SAIBA MAIS

Para complementar seus estudos sobre os sistemas Linux e Windows, leia o artigo a seguir: **Windows vs Linux: compare os recursos dos sistemas para PC em 2020**. Disponível em: <https://globo/3hEuWEZ>. Acesso em 21 jun. 2021.

1.4 MAC OSX

O primeiro sistema operacional desenvolvido para computadores Macintosh foi chamado de System 1. Esse sistema era residente na memória ROM do computador e trabalhava em modo monotarefa. As versões atuais trabalham com multitarefa preemptiva, onde os softwares utilizam o processador, conforme a necessidade e se necessário, interrompem outras tarefas que possuem menor prioridade.

Em 1999, a versão System 9, já era chamada Mac OS. Com melhorias, em 2002 surgiu o Mac OSX, este X era a referência da versão número 10 em algarismo romano.

O sistema Mac OSX, foi desenvolvido com base no NeXTSTEP, que era um sistema operacional Unix baseado no Mach. Esse sistema operacional se popularizou com o uso de interface gráfica.

Este sistema se destaca em relação ao Windows, devido à forma como gerencia os aplicativos abertos, pois ao invés de fechar o programa ao clicar no X como é feito na janela do Windows, ele mantém rodando em segundo plano, consumindo pouca memória e energia, isso facilita na hora de abrir o programa novamente.

Outra vantagem em relação ao Windows, é que não há necessidade de instalar drivers a cada novo periférico conectado via USB, como por exemplo, mouse, teclado, entre outros.

A atualização feita em 2020, chamada de Big Sur, é considerada a maior desde o Mac OSX, trazendo reformulações visuais, aplicativos atualizados, entre outras novidades. Esta versão procura deixar o design mais claro e limpo. A barra de menu superior, por exemplo, também traz um visual mais leve.



LEITURA

Faça a leitura do artigo: **Diferenças entre um PC e um MAC**, e saiba mais sobre os distintos sistemas operacionais.

Disponível em: <https://bit.ly/3r8j3dH>. Acesso em 21 jun. 2021.

1.5 Z/OS

O sistema operacional z/OS, é um sistema operacional que foi criado pela IBM para ser utilizado em mainframes. Um mainframe é um computador de grande porte que pode processar grandes volumes de dados com alta performance, possuindo a característica de escalabilidade e também de segurança. Ele é o sistema operacional mais utilizado em mainframes.

O sistema z/OS possui diversos atributos de outros sistemas operacionais, porém conserva funcionalidades criadas nas décadas de 1960/1970. É possível executar Java de 64 bits, possui suporte a APIs e também, aplicativos Unix, comunicando-se diretamente com o TCP/IP. Desde 2007, este sistema passou a ter suporte somente para mainframes de 64 bits.

Este sistema descende do sistema utilizado na década de 1960 para gerenciar o IBM System360. Nele é possível utilizar Java e executar o Linux.

Em sua versão V2.4, ajuda as organizações a atender os requisitos de nível de serviço e fornece a capacidade de trabalhar com vários volumes de trabalhos diferentes, podendo atender de forma simultânea, diversas necessidades de usuários.



SAIBA MAIS

Faça a leitura da entrevista que o Portal Serpro fez com os especialistas em mainframe, Giancarlo Rodolfi e Carlos Afonso Massiere, ambos da IBM, para compreender o que são mainframes e como eles ainda são utilizados.

Mainframe: o que é e qual o futuro desta tecnologia?

Entrevista disponível em: <https://bit.ly/3wFycUQ>. Acesso em 21 jun. 2021.



2. SISTEMAS OPERACIONAIS REDUZIDOS

2.1 IOS

O sistema operacional iOS vem da sigla *Internet Operating System*. Ele foi desenvolvido para dispositivos móveis com tela touch screen. Ele possui diversos aplicativos nativos da Apple e sua segurança possui um rígido controle de qualidade nos aplicativos que são disponibilizados na App Store. Algumas características são o design minimalista, facilidade de operação e experiência de uso padronizada.

Este sistema deriva do Mac OSX que foi concebido tendo o Unix como base, como vimos anteriormente.

O iOS é mais limitado em questão de personalização se comparado com o Android. Como vantagens podemos citar a leveza, o design, a velocidade, padronização, simplicidade, por outro lado, é pouco flexível e o custo é alto.

A partir da quarta versão, o sistema operacional passou a ser multitarefa permitindo melhor desempenho. O iOS é um sistema embarcado, pago e não possui código aberto. A sua estrutura é dividida em quatro camadas, de acordo com a Apple: Core OS, Core Services, Media e Cocoa Touch, esta é a camada de maior nível.

Vejamos um pouco mais sobre cada camada citada acima:

- » A camada Core OS é a mais próxima do kernel que inclui acesso direto a funcionalidades de baixo nível, gerenciando o sistema operacional, gerenciamento de memória, bateria, segurança, etc.;
- » A camada Core Services é a responsável pelos serviços fundamentais utilizados pelos aplicativos, por exemplo, acesso à agenda, calendário, entre outros;
- » A camada Media possui frameworks de reprodução e gravação de áudio e vídeo, bibliotecas gráficas aceleradas por hardware para animação e renderização 2 e 3D;
- » A camada Cocoa Touch, possui a infraestrutura disponibilizada para implementação da interface visual das aplicações. Trata das interações com o usuário.

O iOS possui alguns aplicativos nativos, como o iTunes, iMovie, entre outros. Na sua tela inicial podemos visualizar diversos ícones e atalhos, o que facilita a navegação do usuário. Também é possível criar pastas com ícones de aplicativos, para criá-las você arrasta um ícone de um *app* sobre outro, deixando-os no mesmo bloco.

A central de notificações pode ser acessada quando arrastamos o dedo de cima para baixo na tela de dispositivos. Assim, também é possível, organizá-las e permitir que usuários possam controlar suas notificações. Na central de controle, podemos acessar outros recursos como Bluetooth, Wi-Fi, brilho, etc.

Podemos utilizar comandos de voz por meio de uma “conversa” com a Siri, a assistente virtual do iOS. Esta assistente é um processo incluso como parte do sistema operacional, o que permite que realize diversas funções, como por exemplo: realizar pesquisas, abrir aplicativos, acessar a agenda, etc.

2.2 ANDROID

O sistema operacional Android foi desenvolvido pela empresa Android Inc. e posteriormente foi comprado pelo Google que se uniu a outras empresas, formando o Open Handset Alliance, responsáveis pelo desenvolvimento contínuo do sistema. O núcleo do Android é baseado em Linux, porém foi personalizado e possui drivers para monitor, câmera, teclado, entre outros dispositivos. A maior parte do desenvolvimento do Android que temos atualmente foi desenvolvida sob a administração do Google.

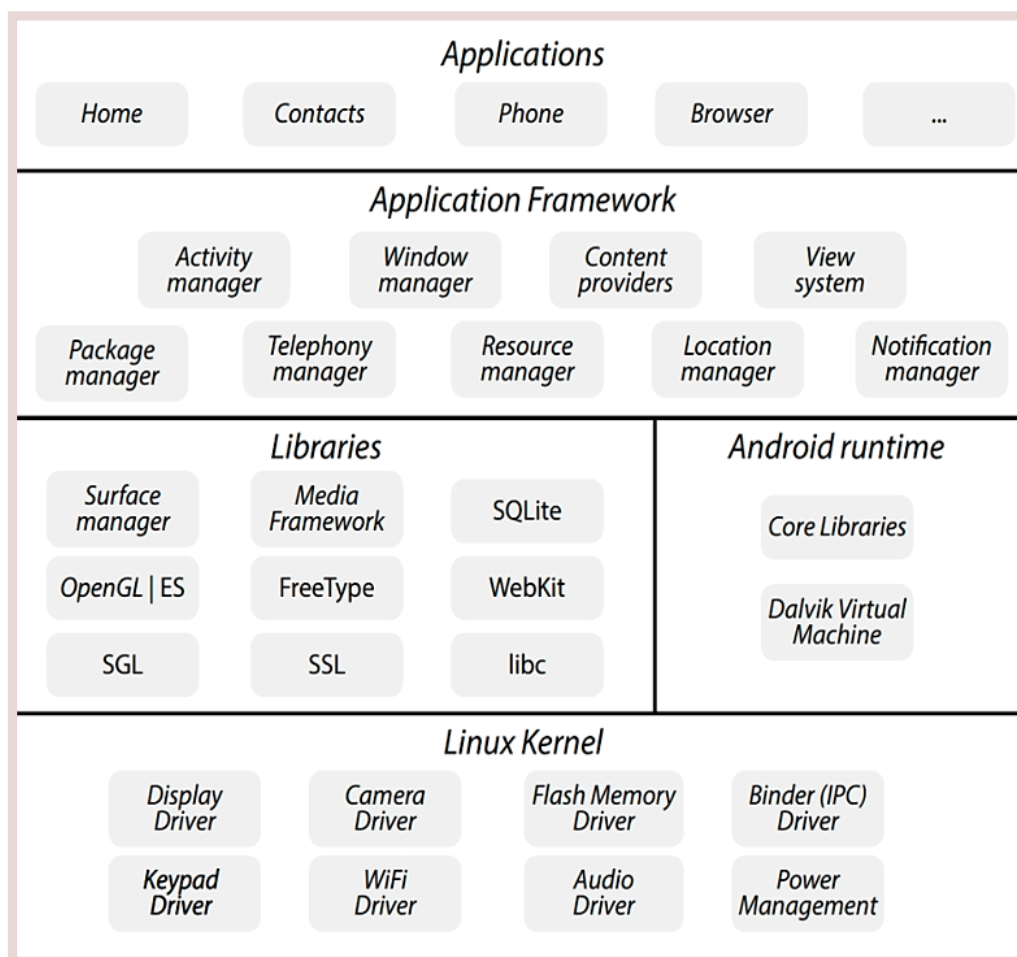
As versões eram denominadas pelos nomes de sobremesas em inglês, como: Cupcake (Android 1.5), Donut (1.6), Eclair (2.0), Froyo (2.2), Gingerbread (2.3), Honeycomb (3.0), Ice Cream Sandwich (4.0), Jelly Bean (4.1), KitKat (4.4), Lollipop (5.0), Marshmallow (6.0), Nougat (7.0), Oreo (8.0) e Pie (9.0). Já o Android 10, deixou de ser denominado desta forma.

Este sistema foi projetado para ser utilizado em dispositivos móveis, como smartphones e tablets. Ele possui alguns recursos integrados como: bluetooth, Wi-Fi, câmera, GPS, entre outros.

A cada dia aumentam a quantidade de aplicativos para Android. Para o desenvolvimento de aplicativos temos ambientes de desenvolvimento gráfico integrado (IDEs), como o Eclipse, onde é possível desenvolver aplicativos utilizando a linguagem Java. Sua arquitetura possui 4 camadas básicas.

A camada de mais alto nível é a de aplicação e aqui ficam os aplicativos. Depois vem a camada de framework com mecanismos de programação para desenvolvimento de aplicativos. Em seguida, a camada com as bibliotecas, que possuem recursos de sistema como GPS, interface gráfica, entre outros. Na camada do Android Runtime tem a garantia de que as threads do sistema possam rodar a sua própria máquina virtual (CÓRDOVA JUNIOR; LEDUR; MORAIS, 2018). A seguir vemos as camadas apresentadas na imagem.

FIGURA 2 - ESTRUTURA DE CAMADAS DO SISTEMA OPERACIONAL ANDROID



Fonte: Córdova Junior, Ledur e Moraes (2018, p. 181).

O Android ficou popular nos dispositivos móveis, mas atualmente também é usada em televisões, sistemas de jogos, relógios inteligentes, entre outros. Parte do Android é escrita em Java (linguagem de alto nível) e o núcleo com um grande número de bibliotecas é escrito em C e C++ (baixo nível).

O Android tem como objetivo dar suporte a um ambiente de aplicações de terceiros, o qual exige uma implementação e API estáveis, para aplicações executarem sobre ele. O Android permite que serviços proprietários sejam criados sobre a plataforma de código aberto.

A interface padrão no Android tem sua base na manipulação direta e os dispositivos que utilizam este sistema operacional são direcionados à tela inicial, assim que ligados. A interface basicamente é composta por ícones de aplicativos e widgets, os quais mostram as informações na tela inicial, como por exemplo, um widget de previsão do tempo.

O sistema operacional Android foi desenvolvido para gerenciar a memória RAM, pois em geral, são usados em dispositivos móveis que dependem do uso de baterias. Ele gera aplicações na memória de forma automática, pois se ela está baixa o sistema elimina aplicativos e processos inativos há algum tempo.

Em relação aos sistemas operacionais para dispositivos móveis, o Android tem uma participação bem maior que os outros sistemas. Mas, com uma grande base de usuários e participação massiva no mercado, é um sistema que fica mais vulnerável a ataques de malware. Como ele possui código aberto, determinadas alterações podem deixar maior vulnerabilidade de segurança, sendo mais fácil sofrer ataques de hackers por exemplo.



SAIBA MAIS

Leia o texto disponível no link a seguir e conheça outros sistemas operacionais para dispositivos móveis, além do Android e IOS. Acesse: <https://glo.bo/3B0u1GB>. Acesso em 21 jun. 2021.



REFLITA

Quais as principais características de cada sistema operacional estudado?

Faça um comparativo entre eles e aproveite para pesquisar mais sobre os sistemas operacionais que estudamos nesta Unidade.

CONSIDERAÇÕES FINAIS

Após realizarmos nossos estudos sobre os principais sistemas operacionais existentes atualmente no mercado, foi possível entender um pouco sobre seu funcionamento.

No dia a dia, vamos dominando mais aqueles sistemas que tivermos maior contato tanto para uso pessoal quanto profissional. Os comandos e configurações vão sendo dominados no uso diário e isso possibilitará compreender melhor ainda seu funcionamento.

Como vimos, o Windows é o sistema operacional mais utilizado em computadores pessoais e o Android em dispositivos móveis. Comparando o Windows com o Mac OSX, pudemos perceber que o este tem melhor desempenho e segurança em relação ao Windows, porém devido a uma série de fatores, inclusive custo dos equipamentos, o Windows permanece sendo muito mais utilizado que o Mac OSX.

Já no caso dos dispositivos móveis, o Android está em primeiro lugar em utilização, além de ser um sistema de código aberto, o que facilita seu acesso e utilização por desenvolvedores. Também, é utilizado em outros tipos de dispositivos, como sistemas de GPS em carros, smart TVs e outros equipamentos eletrônicos.



ANOTAÇÕES

.....

.....

UNIDADE

04

MÁQUINAS VIRTUAIS, COMANDOS SHELL E IMPLEMEN- TAÇÃO DE SCRIPTS



OBJETIVOS DE APRENDIZAGEM

- » Conhecer o funcionamento de máquinas virtuais;
- » Compreender como funciona a prática de comandos em shell;
- » Conhecer a estrutura dos sistemas operacionais.



VÍDEOS DA UNIDADE



<https://bit.ly/3kPLHPu>



<https://bit.ly/3hY6tKV>



<https://bit.ly/3iFjjgz>

INTRODUÇÃO

Nesta Unidade vamos conhecer como funcionam e o que são as máquinas virtuais (VM – *Virtual Machine*) e ainda será possível compreender como a virtualização de sistemas operacionais e a utilização de máquinas virtuais pode ajudar, por exemplo, desenvolvedores a realizarem testes em determinados ambientes ou ainda, em diferentes tipos de plataformas.

Uma máquina virtual é um software no qual é permitido utilizar um computador como se estivesse dentro de outro computador. As VMs se comportam como os computadores físicos, porém você pode simular diversos ambientes sem a necessidade de utilizar uma máquina física com outro tipo de sistema operacional, por exemplo.

Vale a pena comentar que as máquinas virtuais, um dos pontos importantes desta Unidade, não são uma novidade, elas já são utilizadas há vários, porém cada vez mais se tornam úteis para que diversos sistemas possam ser simulados e também, diversas aplicações possam ser testadas.

Na sequência, vamos abordar o que são os comandos em shell e sobre a implementação de scripts. No Linux, temos o Shell Script, que é uma linguagem de script que já está embutida em terminais que utilizam este tipo de sistema operacional e por meio desta linguagem podemos executar comandos diretamente no sistema operacional. Também vamos conhecer sobre os comandos .bat no Windows, que é um conjunto de comandos que são executados em lote de forma sequencial. Ele permite automatizar algumas tarefas rotineiras.

E por fim, outro tema que vamos estudar, é a respeito da estrutura dos sistemas operacionais, onde será possível conhecermos sobre as funções do núcleo e também, sobre as arquiteturas de sistemas operacionais, tais como: arquitetura monolítica, arquitetura de camadas, máquina virtual e arquitetura microkernel.

1. MÁQUINAS VIRTUAIS

Uma máquina virtual ou também conhecida pela sigla VM (*Virtual Machine*) é um nível entre o hardware que está numa camada de nível mais baixo e o sistema operacional. Uma máquina virtual possui uma cópia virtual do hardware, inclusive com os dispositivos de entrada e saída, sendo cada máquina virtual criada independente de outras.

Ela simula um ambiente computacional, que pode incluir hardware, sistemas operacionais, dispositivos de armazenamento, placas de interface de rede, drives, entre outros. Cada VM pode executar diferentes sistemas operacionais ou aplicativos de forma simultânea.

Virtualizar um servidor significa compartilhar uma máquina física onde teríamos diversos servidores virtuais.

Para Córdova Junior; Ledur e Moraes (2018), é importante compreender que a máquina real é o hardware e este fornece acesso aos recursos, os quais vão permitir o funcionamento do sistema operacional. E a VM seria uma fatia da máquina real que possui um sistema operacional específico e isolado do sistema operacional que está na máquina real.

1.1 BREVE HISTÓRICO

Nos anos 1960, nos grandes computadores se gerenciavam os processos de forma manual e neste caso, tinha-se uma demora maior no processamento de operações, assim para que ocorresse um processamento paralelo surgiu a ideia de tempo compartilhada seguida da virtualização. Em 1972 a IBM utilizou pela primeira vez de forma comercial o termo relacionado às máquinas virtuais, ainda em seus mainframes. É um tipo de sistema que ainda existe e passou por uma evolução até a atualidade.

O VM/370 da IBM possibilitava dividir um mainframe em diversas máquinas virtuais e cada qual poderia executar seu sistema operacional. Um problema que existia era em relação aos sistemas de disco, onde se uma máquina física tivesse 3 drives de disco e precisasse suportar 7 máquinas virtuais, ela não poderia alocar um drive de disco para cada VM. Desta forma, uma solução era utilizar discos virtuais (minidiscos) no sistema operacional VM da IBM. Esses minidiscos eram iguais aos discos rígidos, exceto pelo tamanho. Durante muito tempo, a virtualização permaneceu sob o domínio da IBM, pois a maioria dos sistemas não suportava a virtualização (SILBERSCHATZ, 2015).

Somente no final dos anos 1990 quando os processadores da Intel tinham se tornado mais comuns, os desenvolvedores iniciaram diversos esforços para implementar a virtualização nesta plataforma. Desde este período, a virtualização se expandiu para incluir todas as CPUs comuns (SILBERSCHATZ, 2015).

Por volta do ano de 2005, tanto a Intel quanto a AMD, começaram a utilizar o conceito de virtualização em suas máquinas.

1.2 APLICAÇÕES PARA MÁQUINAS VIRTUAIS

Algumas aplicações para máquinas virtuais, segundo Machado e Maia (2017), são: portabilidade de código, consolidação de servidores, aumento da disponibilidade, facilidade de escalabilidade e balanceamento de carga e facilidade no desenvolvimento de software. Vamos conhecer sobre cada uma delas:

Portabilidade de código: como um programa em linguagem de máquina tem necessidade de ser executado em um hardware específico, se ele for gerado em uma linguagem intermediária e utilizar uma máquina virtual para traduzir os comandos para a plataforma onde está sendo executado, esta aplicação pode ser portada para qualquer ambiente sem que precise reescrever todo o programa.

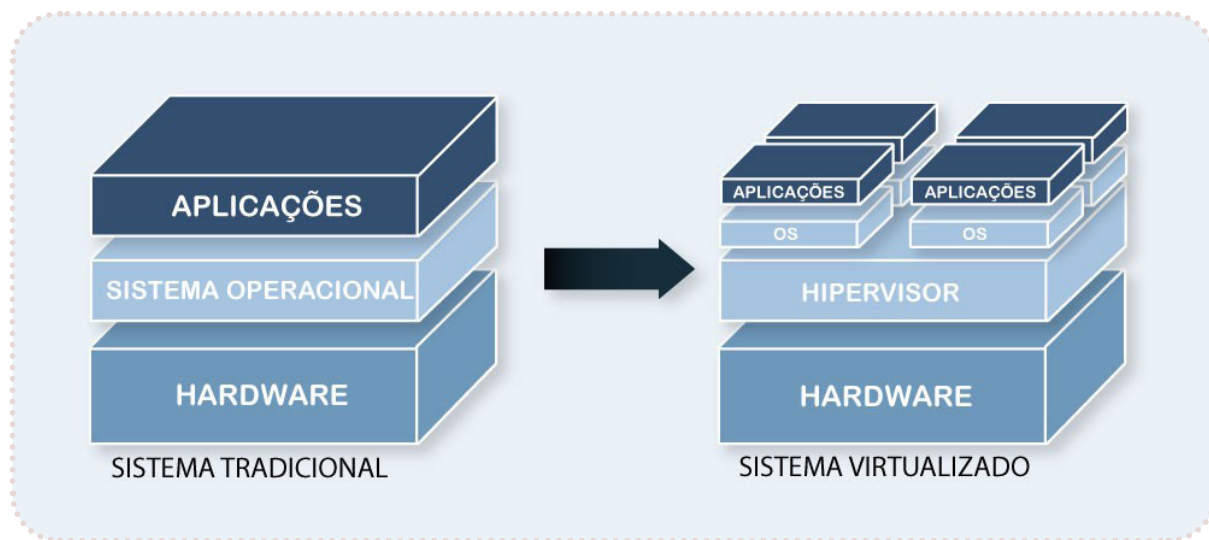
Consolidação de servidores: esta ideia permite que aplicações que estariam sendo executadas em diferentes servidores (com sistemas operacionais e hardwares diferentes), sejam executadas em um único computador usando uma máquina virtual para cada uma das aplicações.

Aumento da disponibilidade: cada máquina virtual pode ser copiada para a memória secundária, assim se houver algum problema com uma delas é possível restaurar a cópia da máquina virtual em outro servidor.

Facilidade de escalabilidade e balanceamento de carga: se acontecer sobrecarga no sistema no qual a máquina virtual está sendo processada e prejudique o desempenho, é possível migrar para outro ambiente com maior capacidade de processamento.

Facilidade no desenvolvimento de software: com as máquinas virtuais é possível criar um ambiente independente para desenvolver e testar softwares sem comprometer os computadores de produção. Também permite, testar um software em diferentes sistemas operacionais.

FIGURA 1 – SISTEMA TRADICIONAL X SISTEMA VIRTUALIZADO



Fonte: QNAP (2021).

Uma VM executa vários processos e o núcleo do sistema operacional define os dispositivos de entrada/saída que podem existir. Uma VM de processo existe de forma temporária e uma VM de sistema é duradoura. Um processo ou sistema operacional que é executado sobre uma VM é chamado de hóspede/convidado e a plataforma onde a VM é executada é chamada de hospedeira/sistema nativo.

Para Alves (2014), uma tarefa importante de um sistema operacional é em relação à administração de recursos, que envolve o controle de entrada e saída de dados, alocação de memória, gerenciamento do processador, acesso ao disco para leitura/escrita, entre outros. Tal gerenciamento ocorre de acordo com as solicitações, como por exemplo, assim que um programa solicita a impressão, o sistema operacional irá alocar espaço na memória, para servir como buffer de impressão, assim permite que o processo ocorra de forma mais rápida.

Em relação ao gerenciamento de memória, quando pensamos em desempenho, nos remetemos ao uso mais adequado da memória em sistemas operacionais, já em ambientes virtualizados temos mais usuários para a memória, ou seja, temos convidados, aplicações e VMM. Neste caso, é necessário que o gerenciamento de memória, ocorra de forma mais eficiente.

Na área de desenvolvimento de sistemas um problema que encontramos é a incompatibilidade entre aplicativos e sistemas operacionais e entre os próprios aplicativos. A virtualização permite armazenar os aplicativos em servidores virtuais, assim quando um usuário precisa acessar um aplicativo o seu processamento pode ser feito pela VM e transmitido para o desktop por meio da rede. Desta forma, é possível deixar o problema de compatibilidade de lado.

Quando utilizamos ambientes virtualizados é necessário nos preocuparmos em considerar a área de gerenciamento de armazenamento. Hipervisores do tipo 0 em geral, permitem o particionamento do disco raiz, porque normalmente executam menos convidados que outros sistemas. Hipervisores do tipo 1 vão armazenar o disco raiz do convidado em um ou em vários arquivos dentro dos sistemas de arquivos e os do tipo 2, vão armazenar as mesmas informações dentro dos sistemas de arquivos do sistema operacional hospedeiro.

O hipervisor que citamos acima, nada mais é que uma camada de software que fica entre o hardware e as máquinas virtuais. Ele é o responsável por fornecer os recursos, tais como memória, processador, entre outros, da máquina física para a VM.

1.3 VANTAGENS E DESVANTAGENS DE UMA VM

1.3.1 VANTAGENS

Na virtualização há a possibilidade de compartilhar o mesmo hardware e executar diversos sistemas operacionais de forma concorrente.

O sistema hospedeiro fica protegido, assim como uma máquina virtual é protegida das outras. Uma VM fica isolada das outras e isso gera uma proteção maior, por exemplo, no caso de um vírus que poderia afetar um sistema operacional convidado e não afetaria os outros convidados ou o próprio hospedeiro.

Com o uso de VMs é possível definir o melhor ambiente para executar um processo, assim caso algum deles possua uma vulnerabilidade não prejudicará os demais.

Uma máquina virtual é encapsulada no VMM (Monitor de Máquina Virtual), o que facilita a troca de plataforma para que seu desempenho possa ser maior.

Permite suporte a aplicações legadas, no caso da utilização de um novo sistema operacional, podemos manter o antigo sendo executado em uma VM, assim podemos diminuir o custo.

Conforme a solução de virtualização usada, pode ser mais fácil monitorar os processos em execução, pois todos são realizados de forma centralizada.

Podemos utilizar um VM como um ambiente de testes, onde é possível testar e avaliar um novo sistema ou uma atualização em um sistema já existente antes de implementá-lo.

Podemos considerar que um sistema de uma VM é excelente para a pesquisa e também para o desenvolvimento de sistemas operacionais.

Como as máquinas virtuais permitem diversos sistemas operacionais sendo executados de forma concorrente, os desenvolvedores podem testar de forma rápida seus programas em diversos ambientes, o que também permite testar diferentes versões, cada uma em um sistema operacional isolado.

No caso do Linux, podemos utilizar um software hospedeiro, como por exemplo, o Virtual Box ou o VMware. Esses dois softwares são gratuitos e de fácil instalação. Com ele instalado, podemos instalar os sistemas operacionais chamados convidados.

1.3.2 DESVANTAGENS

A questão do isolamento pode não permitir o compartilhamento de algum recurso.

Em relação ao quesito segurança, apesar das VMs estarem separadas das outras, elas podem ser menos seguras se comparadas com máquinas físicas.

Ao introduzir uma camada extra de software entre o sistema operacional e hardware (virtualização), pode se ter um custo de processamento superior sem a virtualização, o que diminuiria o desempenho.

Em algumas situações, migrar de um VM para outra pode ser um problema, neste ponto a portabilidade fica prejudicada. Já em relação aos custos de manutenção, treinamento e mão de obra, podem surgir alguns não previstos inicialmente.

1.4 VIRTUALIZAÇÃO

Utilizando o recurso de virtualização podemos rodar diversas máquinas virtuais em uma única máquina física. Esta máquina física irá compartilhar todos os seus recursos utilizando ambientes virtuais. Assim é possível instalar e rodar diferentes sistemas operacionais e aplicações isoladamente em apenas uma máquina física.

Ao utilizarmos o recurso de virtualização, devemos levar em consideração que os recursos disponíveis para cada VM dependem da configuração do hardware, como por exemplo, espaço em disco, memória RAM, entre outros. Dependendo da necessidade do projeto, pode ser mais interessante a empresa investir na compra de hardware já projetado como servidor de virtualização, assim ele já possui as características necessárias para um maior desempenho.

Além do hardware, é necessário um sistema operacional para servir como hospedeiro das VMs e este sistema operacional é chamado de hipervisor ou Monitor de Máquina Virtual (VMM). O hipervisor pode definir o que será virtualizado, quais os recursos serão alocados. O hipervisor controla o acesso dos sistemas operacionais visitantes em hardware.

Os hipervisores do tipo 1 são executados diretamente sobre o hardware e os do tipo 2 podem utilizar os serviços e abstrações oferecidos pelo sistema operacional subjacente.

Segundo Rosenblum (2004 apud CÓRDOVA JUNIOR; LEDUR; MORAIS, 2018), temos 4 atributos que são compartilhados entre as VMs, mesmo que elas sejam diferentes:

- a) Compatibilidade do software: neste caso se tem uma garantia de que um software escrito para a máquina irá funcionar;
- b) Isolamento: garante que a execução dos softwares nas diferentes VMs irão ocorrer de forma isolada;
- c) Encapsulamento: é a manipulação e controle da execução de softwares nas diferentes VMs;
- d) Desempenho: ao colocarmos mais uma camada de software, podemos alterar o desempenho, porém os benefícios da virtualização devem compensar.

Quando se utiliza a virtualização de servidores é possível reduzir o consumo de energia elétrica e também da emissão de carbono na atmosfera, além da tolerância a falhas em sistemas computacionais (CÓRDOVA JUNIOR; LEDUR; MORAIS, 2018).

Um exemplo da utilização da virtualização é a computação em nuvem, onde temos recursos como processador, memória e dispositivos de entrada e saída disponibilizados por meio da internet.

No caso da nuvem, a ideia é que podemos terceirizar a necessidade de computação ou ainda de armazenamento para um local que é administrado por uma empresa que possui *know-how* na área. Dentre as vantagens da computação em nuvem está a que não precisamos nos preocupar com máquinas físicas, manutenção, infraestrutura, pessoal especializado, etc. Além disso, os provedores da nuvem permitem que diversos clientes possam compartilhar uma mesma máquina física.

Quando uma máquina virtual é criada, são fornecidos determinados parâmetros ao VMM, o que inclui muitas vezes o número de CPUs, a quantidade de memória, informações sobre a conexão de rede e sobre o armazenamento que o VMM irá levar em conta quando criar o convidado. Se eu desejar criar um convidado com determinada configuração, o VMM cria a máquina virtual com os parâmetros desejados, de acordo com a disponibilidade.

Se uma VM não for mais utilizada, existe a opção de excluí-la, onde primeiramente o VMM libera o espaço em disco utilizado e depois é removida a configuração associada à VM.

Segundo Tanenbaum e Bos (2016), os hipervisores possuem as seguintes dimensões:

- a) Segurança: o hipervisor precisa ter controle completo dos recursos virtualizados;
- b) Fidelidade: um programa deve se comportar em uma VM da mesma forma que se comportaria sendo executado em uma máquina real;
- c) Eficiência: a maior parte do código em uma VM deve executar sem a intervenção do hipervisor.

Em geral, a virtualização pode ser dividida em virtualização completa e paravirtualização.

1.4.1 VIRTUALIZAÇÃO COMPLETA OU TOTAL

Neste tipo de virtualização, o hipervisor simula o hardware da máquina física e as VMs executam de forma isolada.

Na virtualização total temos uma cópia do hardware subjacente onde o sistema operacional e outras aplicações são executadas como se estivessem no hardware original e permite que o sistema operacional convidado não seja modificado para executar sobre o monitor de máquina virtual (VMM). Porém, pelo fato de executarmos todas as instruções no sistema convidado, precisamos testar na máquina virtual para saber se estas são sensíveis ou não e isto, acaba acarretando um custo de processamento maior.

Outro problema deste tipo de virtualização, é a dificuldade em se implementar uma VM que reproduza o comportamento exato de cada tipo de dispositivo (teclado, mouse, portas, controladores, etc.). Desta forma, podemos ter uma subutilização de um recurso de hardware físico.

Outra situação que pode ocorrer é que uma VM deve contornar problemas relacionados à implementação do gerenciamento de memória.

1.4.2 PARAVIRTUALIZAÇÃO

Já na paravirtualização, ocorre a entrega para as VMs de um hardware igual ao real e possibilita que o sistema que for virtualizado possa sofrer alterações no decorrer do tempo. A paravirtualização tem maior desempenho e pode se adaptar a modificações do sistema operacional.

A paravirtualização utiliza uma abordagem diferente dos outros tipos de virtualização, pois ao invés de tentar fazer com que um sistema operacional convidado acredite que tem um sistema para si mesmo, a paravirtualização apresenta o convidado a um sistema que é semelhante, porém não é igual ao sistema que ele prefere. O convidado precisa ser modificado para que possa ser executado no hardware paravirtualizado, tornando o uso de recursos mais eficiente (SILBERSCHATZ; GALVIN; GAGNE, 2015).

Na paravirtualização o hóspede precisa estar ciente do API da máquina virtual. Assim, ela deve ser customizada explicitamente para o hipervisor.



LEITURA

Para complementar seus estudos, leia o artigo a seguir que fala sobre a virtualização e paravirtualização- Virtualização: VMWare e Xen. Disponível em: <https://bit.ly/2VMoOSV>. Acesso em 23 jun. 2021.

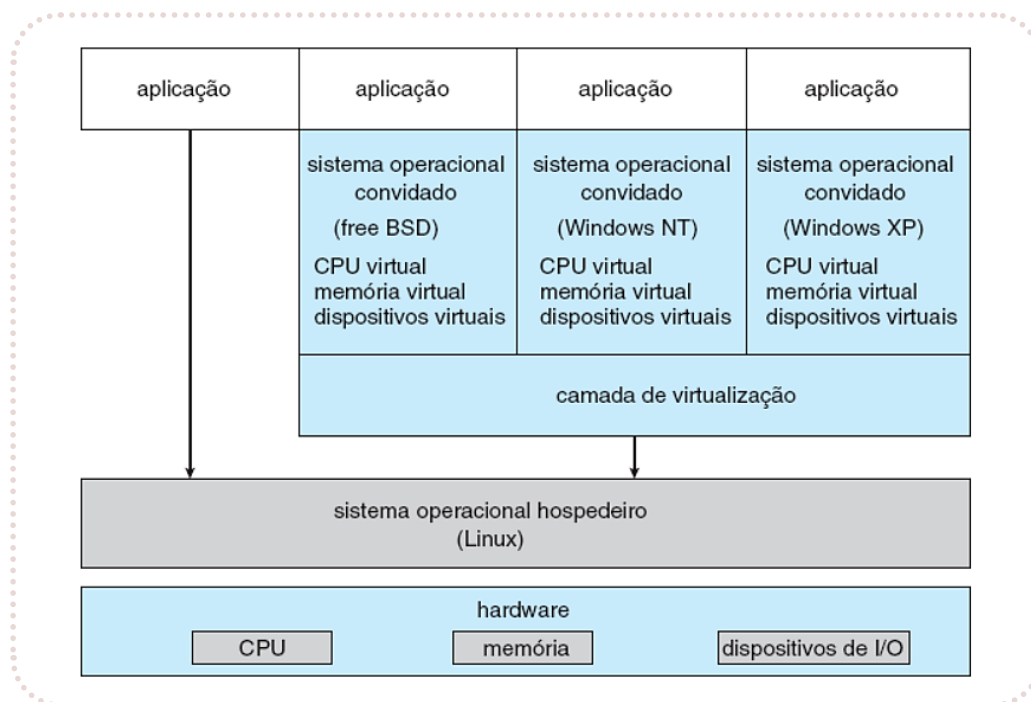
1.5 VMWARE WORKSTATION

Apesar do conceito de máquinas virtuais ser antigo, somente na atualidade, elas têm tido maior atenção e estão se tornando cada vez mais populares, pois permitem resolver problemas como a falta de compatibilidade entre sistemas.

Um dos exemplos que podem ser citados, é o VMware Workstation, que é uma aplicação que abstrai o Intel x86 e o hardware compatível em VMs isoladas. Ele é um exemplo de hipervisor tipo 2. Esta aplicação é executada em um sistema operacional hospedeiro, como por exemplo, Windows ou Linux. Ele permite executar concorrentemente diversos sistemas operacionais convidados diferentes como VMs independentes. Podemos ver a arquitetura deste sistema, na figura 2 (SILBERSCHATZ, 2015).

No cenário apresentado na figura a seguir, o Linux é o sistema operacional hospedeiro, já os sistemas operacionais FreeBSD, o Windows NT e o Windows XP, neste caso, são os sistemas operacionais convidados.

FIGURA 2 – ARQUITETURA DO VMWARE WORKSTATION



Fonte: Silberschatz (2015).

No coração do VMware está a camada de virtualização, a qual abstrai o hardware físico em máquinas virtuais isoladas sendo executadas como sistemas operacionais convidados. Neste caso, cada uma das máquinas virtuais tem sua própria CPU, memória, drives de disco, interfaces de rede virtuais, etc. O disco físico que o convidado possui e gerencia é na verdade um arquivo dentro do sistema de arquivos do sistema operacional hospedeiro. Neste exemplo, para que seja possível criar um convidado idêntico, podemos simplesmente copiar o arquivo. Esta cópia do arquivo para outra localização protege o convidado contra um desastre no local original. Este tipo de cenário mostra como a virtualização pode melhorar a eficiência da administração e o uso de recursos do sistema (SILBERSCHATZ, 2015).

2. LINGUAGEM DE COMANDOS E SCRIPTS

Uma linguagem de comando permite que um usuário possa se comunicar de forma fácil com o sistema operacional, executando diversas tarefas. Essa é uma forma do usuário ter um acesso direto ao sistema operacional. Cada sistema operacional possui uma linguagem de comandos.

Vejam alguns exemplos: DCL - *Digital Command Language* é utilizada no OpenVMS, a JCL - *Job Control Language*, é utilizada no MVS da IBM, comandos do shell estão disponíveis em diversos sistemas Unix. A seguir podemos ver alguns comandos como exemplo de utilização no Microsoft Windows.

QUADRO 1 – EXEMPLOS DE COMANDOS DO MICROSOFT WINDOWS

COMANDO	DESCRIÇÃO
dir	Lista o conteúdo de um diretório
cd	Altera o diretório default
type	Exibe o conteúdo de um arquivo
del	Exclui arquivos
mkdir	Cria diretórios
ver	Mostra a versão do Windows

Fonte: Adaptado de Machado e Maia (2017).

Após a digitação de um determinado comando o shell ou um interpretador de comandos vai verificar se o comando está correto para chamar a rotina e em seguida apresentar o resultado. Em geral os sistemas operacionais atuais, possuem uma interação utilizando interfaces gráficas, como é o caso do Microsoft Windows, o que facilita para o usuário.

Em geral as linguagens de comandos permitem a possibilidade de criar programas que possuem uma estrutura de decisão e iteração. Esses programas possuem uma sequência de comandos e são chamados, por exemplo, de arquivos batch ou shell scripts, que veremos na sequência.



SAIBA MAIS

Comandos Bash Linux – Guia Básico e Exemplos de Uso

Para conhecer sobre os comandos Bash Linux (ferramentas de script do Unix), acesse o link: <https://bit.ly/3i9fyzz>. Acesso em 23 jun. 2021.

2.1 SCRIPTS

Os scripts permitem criar arquivos com tarefas repetitivas que podemos utilizar no nosso dia a dia. Quando falamos de scripts em um sistema operacional, lembramos que os arquivos com o seu código, pode ser identificado pela extensão.

No caso do sistema operacional Windows, a extensão utilizada é .bat e esses arquivos são executados com permissão automática. Também é possível em algumas situações que os scripts sejam criados para serem executados automaticamente utilizando o agendador de tarefas.

No agendador é só utilizar a opção Criar Tarefa Básica. Nele é possível definir a frequência e horário para executar esta tarefa.

Cada linha de comando em um script é lida e interpretada como um comando diferente. Esses comandos são executados em sequência. Para criar um arquivo .bat no Windows, você pode usar o bloco de notas.

Veja um exemplo:

Digite os comandos abaixo usando o bloco de notas.

MD textos

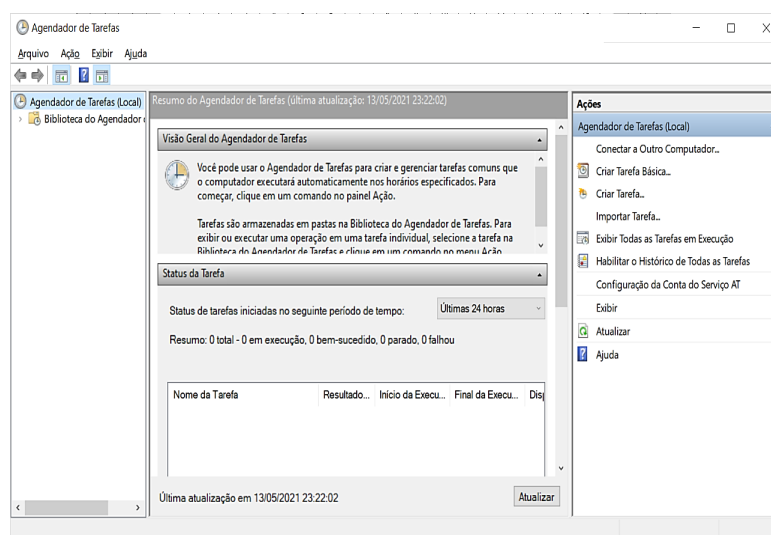
Copy F:\arquivos textos

Salve este arquivo com os comandos. Não esqueça que ele deve ter a extensão .bat. Você pode salvar como ‘copia.bat’, por exemplo.

Esse script ao ser utilizado irá criar uma pasta chamada “textos”, em seguida irá copiar do drive F de uma pasta chamada arquivos os seus arquivos para a nova pasta chamada textos.

Na figura a seguir, podemos ver um exemplo do agendador de tarefas do Windows 10.

FIGURA 3 – AGENDADOR DE TAREFAS DO WINDOWS 10



Fonte: A autora/Windows (2021).

Vamos ver alguns comandos que podem ser úteis no quadro a seguir.

QUADRO 2 – EXEMPLO DE COMANDOS USADOS EM ARQUIVOS .BAT

COMANDO	O QUE FAZ
md	Usado para criar um diretório
copy	Realiza cópia de arquivos
pause	Executa uma pausa
shutdown /s	Desliga o computador
cls	Limpa a tela
echo	Escreve um texto na tela
start	Inicia um programa ou executa um arquivo .exe
move	Movimenta um arquivo de um diretório para outro

Fonte: A autora (2021).

No caso do sistema operacional Linux, a extensão utilizada é `.sh`. e nele, precisamos executar o comando `chmod` para definir a permissão de execução destes arquivos. A execução é feita por meio do terminal de comandos inserindo os caracteres `./` antes do caminho do arquivo. Para agendamento de tarefas no Linux, utiliza-se o comando `crontab`.

Os scripts permitem economizar tempo no caso de tarefas rotineiras. É importante que não esteja logado como `root`, no caso do Linux, pois neste caso, um erro pode causar muitos problemas. Para criação de arquivos de script podemos utilizar o editor de textos `vi`. Após criar o arquivo é necessário definir a permissão de execução do arquivo, por meio do comando `chmod`.

Caso seja necessário adicionar comentários no código, que não são executáveis, basta utilizar o caractere `#` no início da linha. Já se desejar colocar alguma informação na tela, para o usuário, pode-se usar o comando `echo`.

3. ESTRUTURA DOS SISTEMAS OPERACIONAIS

Um sistema operacional possui em sua composição diversas rotinas que são chamadas de núcleo do sistema (`kernel`). Os usuários podem se comunicar com o núcleo do sistema operacional por meio das chamadas rotinas do sistema realizadas por aplicações, também podem interagir com o núcleo por meio de utilitários ou ainda por meio de linguagem de comandos. Tais utilitários ou linguagem de comandos é específica de cada sistema operacional.

Machado e Maia (2017) citam algumas das funções do núcleo dos sistemas operacionais, sendo que diversas delas vimos no decorrer da disciplina:

- a) tratamento de interrupções e exceções;
- b) criação e eliminação de processos e threads;
- c) sincronização e comunicação entre processos e threads;
- d) escalonamento e controle dos processos e threads;
- e) gerência de memória;
- f) gerência do sistema de arquivos;
- g) gerência de dispositivos de E/S;
- h) suporte a redes locais e distribuídas;
- i) contabilização do uso do sistema;
- j) auditoria e segurança do sistema.

Quando trabalhamos com um sistema multiprogramável, podem surgir alguns problemas em relação à segurança, pois temos vários usuários compartilhando recursos da máquina, como por exemplo, memória, processador, etc. Por este motivo, é necessário que um sistema operacional possua confiabilidade na execução concorrente dos aplicativos, garantindo também sua integridade.

É necessária a preocupação em relação a formas de proteger o núcleo do sistema operacional e também, o acesso aos seus serviços, pois determinadas ações, podem comprometer sua integridade.

Normalmente os processadores permitem dois tipos de acesso, o acesso em modo usuário e o acesso em modo kernel. No modo usuário, uma aplicação só pode executar instruções conhecidas como não privilegiadas, tendo acesso a um número reduzido de instruções. No modo kernel, a aplicação pode ter acesso ao conjunto total de instruções do processador. Por exemplo, se uma aplicação tiver acesso a determinadas áreas de memória onde está carregado o sistema operacional, um desenvolvedor poderia de forma mal intencionada ou ainda por erro no código, acabar gravando nesta área e isso causaria uma violação no sistema (MACHADO; MAIA, 2017).

Quando um sistema operacional é projetado é necessário levar em consideração alguns requisitos, como: confiabilidade, flexibilidade, portabilidade, desempenho, entre outros. Quando um sistema operacional é projetado, utilizando linguagens de alto nível, isso facilita a alteração em outra arquitetura de hardware, porém pode haver perda de desempenho. Sendo assim, as partes mais complexas podem ser programadas em assembly. Mas, atualmente existe uma tendência de utilização de técnicas de orientação por objetos.

Para Machado e Maia (2017), alguns dos benefícios de utilizar a orientação por objetos no desenvolvimento de um projeto de sistemas operacionais, são: mais organização das funções e também dos recursos do sistema, redução no tempo de desenvolvimento do sistema, aumentar a facilidade para realizar a manutenção e extensão do sistema, melhorias no processo de implementação do modelo de computação distribuída.

Para compreendermos melhor como a estrutura do núcleo de um sistema operacional é organizada e também como se dá o inter-relacionamento dos seus componentes, vamos abordar algumas das principais arquiteturas de sistemas operacionais, citadas por Machado e Maia (2017) e por Taneubaum e Bos (2016), tais como: arquitetura monolítica, arquitetura de camadas, máquina virtual e arquitetura microkernel.

Arquitetura monolítica: os módulos são compilados de forma separada e posteriormente são *linkados*, assim eles formam um programa executável grande onde os módulos interagem entre si livremente. Este modelo fazia com que os sistemas operacionais fossem difíceis de receber manutenção. Essa estrutura foi utilizada no desenvolvimento do MS-DOS e também, dos primeiros sistemas operacionais UNIX.

Este modelo sugere uma estrutura para o sistema operacional: um programa principal faz a chamada a uma rotina de serviço que foi requisitada, um conjunto de rotinas de serviço executam as chamadas de sistema, um conjunto de rotinas utilitárias que ajudam as rotinas de serviço (TANEUBAUM; BOS, 2016).

Arquitetura de camadas: à medida que os sistemas foram ficando mais complexos e maiores, foi necessário incorporar técnicas de programação estruturadas e modulares. Nesta arquitetura o sistema é dividido em níveis sobrepostos, onde cada camada disponibiliza um conjunto de funções e estas podem usadas pelas camadas superiores. Este modelo de arquitetura facilita a manutenção e depuração e ainda cria uma hierarquia de níveis de modos de acesso, porém tem menor desempenho. A maioria das versões de UNIX e também o Windows, estão baseados nesta arquitetura.

Máquina virtual: o modelo de *virtual machine* (VM), cria um nível intermediário entre hardware e sistema operacional. Neste nível são criadas máquinas virtuais independentes, desta forma, cada VM pode ter seu próprio sistema operacional, como já vimos anteriormente.

Arquitetura microkernel: atualmente uma característica dos sistemas operacionais é tornar o núcleo menor e o mais simples possível, assim os serviços dos sistemas são disponibilizados por meio de processos. Cada processo é responsável por oferecer um conjunto específico de funções, como por exemplo: gerenciamento de arquivos, de processos, memória, entre outros. Quando uma aplicação vai utilizar um determinado serviço, faz a solicitação ao processo que seja responsável. Esta aplicação é o cliente e a função do núcleo é realizar a comunicação entre cliente e servidor. Somente o núcleo do sistema, executa no modo kernel, assim caso aconteça algum erro em um servidor, ele poderá parar, porém não compromete todo o sistema. Neste modelo de arquitetura, a manutenção é mais fácil. Mas, a implementação é muito difícil, pois existe um problema de desempenho por causa da necessidade de mudança do modo de acesso a cada comunicação entre clientes e servidores. Algumas funções do sistema operacional precisam fazer o acesso direto ao hardware como as operações de entrada e saída, sendo este outro problema neste tipo de arquitetura.

Em geral, o que é implementado é uma combinação do modelo de camadas com a arquitetura microkernel. Neste caso, o núcleo do sistema, além de ficar responsável pela parte de comunicação cliente-servidor, ele passa a incorporar outras funções críticas do sistema, como escalonamento, tratamento de interrupções e gerenciamento de dispositivos (MACHADO; MAIA, 2017).

CONSIDERAÇÕES FINAIS

Nesta Unidade estudamos a respeito das máquinas virtuais, onde foi possível conhecermos um pouco a respeito de seu funcionamento e também, conhecer melhor suas aplicações na área computacional.

Vimos que uma máquina física pode se “transformar” em diversas outras máquinas virtuais permitindo utilizar aplicações em diferentes ambientes virtuais, pode testar desta forma, como determinados sistemas irão se comportar em outro sistema operacional, por exemplo.

Também, foi possível conhecer sobre os comandos em shell voltados ao sistema operacional Linux e os comandos que são executados em lote, do tipo batch, no Windows em suas diversas versões.

Dentro deste contexto, foi possível conhecermos melhor como funciona a estrutura dos sistemas operacionais e como estes podem ser projetados e desenvolvidos.

Esta Unidade permitiu encerrarmos os estudos sobre sistemas operacionais, compreendendo melhor suas aplicações e práticas do dia a dia.

Espero que tenha sido possível aproveitar todos esses conhecimentos, para que futuramente possa aprofundá-los e utilizá-los em sua prática profissional.

Bons estudos!



REFERÊNCIAS

- ALVES, William Pereira. **Sistemas Operacionais**. São Paulo: Érica, 2014.
- CAPRON, H. L. **Introdução à Informática**. Tradução José Carlos Barbosa dos Santos. São Paulo: Pearson Prentice Hall, 2004.
- CÓRDOVA JUNIOR, Ramiro Sebastião; LEDUR, Cleverson Lopes; MORAIS, Izabelly Soares de. **Sistemas operacionais**. Porto Alegre: SAGAH, 2018.
- MACHADO, Francis B.; MAIA, Luiz Paulo. **Arquitetura de sistemas operacionais**. 5. ed. Rio de Janeiro: LTC, 2017.
- _____. **Fundamentos de sistemas operacionais**. Rio de Janeiro: LTC, 2011.
- OLIVEIRA, Rômulo Silva de; CARISSIMI, Alexandre da Silva; TOSCANI, Simão Sirineo. **Sistemas operacionais**. 4. ed. (Série Livros Didáticos Informática, n. 11). Porto Alegre: Bookman: Instituto de Informática da UFRGS, 2010.
- PEREIRA, Adriana Soares; VISSOTTO, Elisa Maria; FRANCISCATTO, Roberto. **Sistemas operacionais**. Frederico Westphalen: Universidade Federal de Santa Maria, Colégio Agrícola de Frederico Westphalen, 2015. Disponível em: https://www.ufsm.br/app/uploads/sites/413/2018/12/arte_sistemas_operacionais.pdf. Acesso em: 25 fev. 2021.
- SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. **Fundamentos de sistemas operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.
- TANENBAUM, Andrew S.; WOODHULL, Albert S. **Sistemas operacionais: projeto e implementação**. 3. ed. Porto Alegre: Bookman, 2008.
- TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas operacionais modernos**. Tradução Jorge Ritter. 4. ed. São Paulo: Pearson Education do Brasil, 2016.
- QNAP. **O que é Virtualização?**, 2021. Disponível em: <https://www.qnapbrasil.com.br/blog/post/o-que-e-virtualizacao>. Acesso em 10 mai. 2021.

